

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА №1

Основы криптосистемы RSA

по дисциплине
«Криптографические методы защиты информации»

Выполнил
студент гр. 5131001/20502

Улановский Г. А

Преподаватель
ассистент ВШК ИКНК

Иванов М. С.

1 ЦЕЛЬ РАБОТЫ

Изучить принципы криптосистемы RSA, гибридного шифрования RSA + AES-256-CBC, структурированного хранения данных в ASN.1 DER-контейнерах и электронной подписи RSA/SHA-256, а также продемонстрировать непригодность CRC32 для криптографической подписи.

2 ЗАДАЧИ РАБОТЫ

В ходе выполнения лабораторной работы необходимо разработать программу П-1, выполняющую зашифрование и расшифрование файла с использованием RSA и AES-256-CBC, а также формирование и проверку электронной подписи RSA/SHA-256. В дополнительной части необходимо реализовать программу П-2 для подписи RSA/CRC32 и программу П-3, формирующую документ D3, который содержит данные D2, но проходит проверку старой подписью D1.

- сгенерировать параметры криптосистемы RSA и ключ AES-256;
- зашифровать файл по гибридной схеме и сохранить результат в ASN.1 DER-контейнере;
- расшифровать контейнер и проверить совпадение с исходным файлом;
- сформировать и проверить подпись RSA/SHA-256;
- сформировать подпись RSA/CRC32 и показать возможность подделки за счёт подбора четырехбайтного дополнения.

3 ХОД РАБОТЫ

В ходе выполнения лабораторной работы была разработана программа на языке Python, включающая модули генерации ключей RSA, гибридного шифрования файла, формирования и проверки электронной подписи, кодирования ASN.1 DER-контейнеров и демонстрации уязвимости подписи на основе CRC32. Общая структура разработанной программы представлена на рисунке 1.

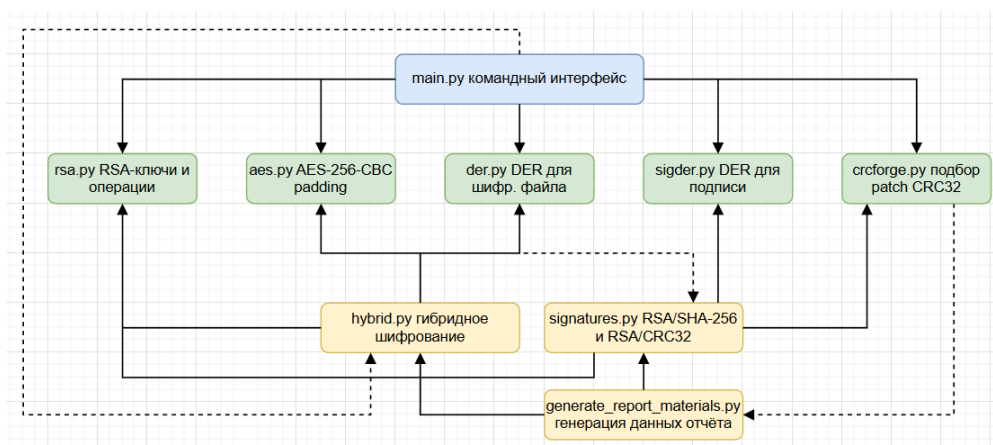


Рисунок 1 – Структура разработанной программы

3.1 Теоретические сведения

3.1.1 RSA

В RSA выбираются два простых числа p и q , после чего вычисляется модуль n :

$$n = p \cdot q. \quad (1)$$

Функция Эйлера для такого модуля вычисляется по формуле:

$$\varphi(n) = (p - 1)(q - 1). \quad (2)$$

Открытый показатель e выбирается взаимно простым с $\varphi(n)$, а закрытый показатель d является обратным к e по модулю $\varphi(n)$:

$$e \cdot d \equiv 1 \mod \varphi(n). \quad (3)$$

Шифрование и расшифрование целого представителя сообщения выполняются по формулам:

$$c \equiv m^e \mod n, \quad (4)$$

$$m \equiv c^d \bmod n. \quad (5)$$

В работе используется учебная реализация RSA для демонстрации принципов. В реальных системах textbook RSA без padding не применяется: для шифрования используют OAEP, для подписи - PSS или другие стандартизованные схемы.

3.1.2 Гибридное шифрование RSA + AES

RSA не используют для шифрования больших файлов напрямую: операция медленная, размер блока ограничен модулем, а без padding схема детерминирована. Поэтому применяется гибридная схема: данные файла шифруются симметричным алгоритмом AES, а RSA шифрует только случайный AES-ключ.

В данной работе файл шифруется AES-256-CBC. Случайный 32-байтный ключ AES помещается в контейнер в RSA-зашифрованном виде. IV нужен для режима CBC, чтобы одинаковые первые блоки открытого текста не давали одинаковые первые блоки шифртекста при одном и том же ключе.

3.1.3 AES-256-CBC

AES-256 использует ключ длиной 256 бит и размер блока 128 бит. Режим CBC связывает каждый блок открытого текста с предыдущим блоком шифртекста через XOR перед шифрованием. Для первого блока используется IV длиной 16 байт. Так как AES работает с блоками фиксированного размера, перед шифрованием применяется PKCS#7 padding.

3.1.4 ASN.1 DER-контейнеры

Структурированный бинарный контейнер нужен, чтобы хранить вместе версию формата, RSA-зашифрованный ключ, IV, шифртекст и параметры подписи без неоднозначного разбора. DER является однозначным бинарным кодированием ASN.1. ASN.1-структуры контейнера зашифрованного файла и контейнера подписи представлены в листинге 1.

Листинг 1 – ASN.1-структуры контейнеров

```
EncryptedFile ::= SEQUENCE {  
    version INTEGER,  
    encryptedKey OCTET STRING,  
    iv OCTET STRING,  
    ciphertext OCTET STRING  
}  
SignatureFile ::= SEQUENCE {  
    version INTEGER,  
    algorithm UTF8String,  
    hashValue OCTET STRING,  
    hashModN INTEGER,  
    signature INTEGER  
}
```

3.1.5 Электронная подпись RSA/SHA-256

При формировании электронной подписи сначала вычисляется хэш сообщения и приводится по модулю n :

$$h \equiv \text{SHA} - 256(M) \bmod n. \quad (6)$$

Затем значение h возводится в степень закрытого показателя d :

$$s \equiv h^d \bmod n. \quad (7)$$

Проверка подписи выполняется с использованием открытого показателя e :

$$s^e \bmod n = h. \quad (8)$$

SHA-256 подходит для лабораторной как криптографическая хэш-функция с большим выходом и практической устойчивостью к подбору коллизий.

3.1.6 CRC32 и причина непригодности для электронной подписи

CRC32 является линейной контрольной суммой для обнаружения случайных ошибок, а не криптографическим хэшем. Выход имеет всего 32 бита, коллизии легко находятся, а линейность позволяет конструктивно изменять данные при сохранении заданного CRC32.

В дополнительном задании используется следующее соотношение:

$$D3 = D2 || \text{patch}, \quad (9)$$

где четыре байта `patch` подбираются так, чтобы

$$CRC32(D3) == CRC32(D1), \quad (10)$$

Следовательно, если подпись построена только по CRC32, то одна и та же подпись может быть принята для двух различных документов D1 и D3.

3.2 Описание реализации

Код разделен на модули: `rsa.py` реализует RSA, `aes.py` — AES-256-CBC и `padding`, `der.py` и `sigder.py` — DER-контейнеры, `hybrid.py` — файловое гибридное шифрование, `signatures.py` — подписи, `crcforge.py` — подбор четырехбайтного CRC32-патча. CLI в `src/main.py` запускает основные операции лабораторной.

3.3 Параметры RSA

Для демонстрационного запуска использовались учебные параметры RSA с модулем длиной 1023 бита, полученным при генерации 1024-битной пары ключей. Параметры ключа приведены в таблице 1.

Таблица 1 – Параметры RSA

Параметр	Значение
p	8917077033301448496395301444247094188183470446590314270663 4322424476807373569989637168067003052604103936213697351172 19341777829130823026844381889395986463
q	7847715146722323202556967262648340985221226465337723999873 2566299719381395140756325265508843505841161858502743468799 85531450815433974119720592128466395831
n = p*q	6997868049872953538904181644247023232279532107212259433752 4206160339612880680971485757603189557505766452171609109332 7268265427348037356302144557707482385670442670388860740607 7520948847240220421610996591407535781761595298088204425326 4068896780336859858695556690088050001797980536496259886416 018554458475635753
e	65537

Таблица 1 – Параметры RSA (продолжение)

Параметр	Значение
d	9819246316833495696135443245875707713817015923512510147365 1921792343726452346340812522226975780219269770384686111574 1880001963759792411699424958829059617785937792090113573278 1844811745758713632631253253209592362106984342300930577332 1084352484188972353677756763478757002950732380640582883092 28300436881143153

3.4 Гибридное шифрование и расшифрование

Для шифрования файла использовалась гибридная схема: содержимое файла шифровалось симметричным алгоритмом AES-256 в режиме CBC, а случайный ключ AES шифровался открытым ключом RSA. Результат сохранялся в ASN.1 DER-контейнере, содержащем зашифрованный ключ, вектор инициализации и шифртекст. Схема гибридного шифрования представлена на рисунке 2.

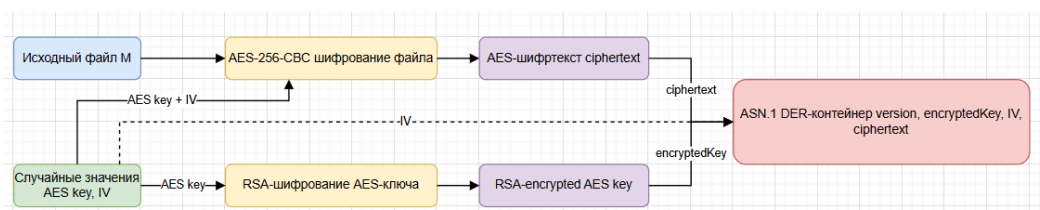


Рисунок 2 – Схема гибридного шифрования файла

Исходный файл data/message.txt был зашифрован по гибридной схеме: содержимое файла шифруется AES-256-CBC, а 32-байтный AES-ключ шифруется RSA. Размер исходного файла составил 84 байта, размер DER-контейнера - 253 байта.

Сгенерированные значения AES-ключа и IV, использованные при шифровании файла, приведены в листинге 2.

Листинг 2 – Сгенерированные AES-ключ и IV

AES-256	key	=
49e91c7779e9cef5c289737b8416be3122b61472c985bd2e28f3aaa721aded1b		

RSA-зашифрованный AES-ключ в шестнадцатеричном виде приведён в листинге 3:

Листинг 3 – RSA-зашифрованный AES-ключ

```
12e26f42c1825270749ad1cc846d9cf4bd8ae72fe8883aadcf6ea4ffeae17d17c
909983420ba031e5e444afacee81722ac30f7e131cab29519041984f9c92d67d2
9ae03c7c07a7e8e9fe92a407d648b56f20cc81ba0cb50d1000c4e9243116e6bbb
4aa35507430595747af2614df691caea8abf3d3fd3b373d1f73e0e43d9b8b
```

Первые 100 байт DER-контейнера зашифрованного файла приведены в листинге 4:

Листинг 4 – Первые 100 байт DER-контейнера

```
3081fa02010104818012e26f42c1825270749ad1cc846d9cf4bd8ae72fe8883aa
dcf6ea4ffeae17d17c909983420ba031e5e444afacee81722ac30f7e131cab295
19041984f9c92d67d29ae03c7c07a7e8e9fe92a407d648b56f20cc81ba0cb50d1
000c4
```

Первые 100 байт AES-шифртекста приведены в листинге 5:

Листинг 5 – Первые 100 байт AES-шифртекста

```
76b3f3ab89abcba341415636d05735e6e0dd6ea7749cbfe93d4dd0dabed5ae00c
e42d8c04cfe1807ec5b6fcd6305487ec9870958fa6949a87076981048c56ffaee
c68e2af2fec555597e3b76580c625dd23fa881cbcae25c6dd85dad0daad9cc
```

При расшифровании программа разбирает DER-контейнер, извлекает RSA-зашифрованный ключ, расшифровывает его закрытым RSA-ключом и затем восстанавливает файл с помощью AES-256-CBC.

В результате расшифрованный файл совпал с исходным, что подтверждает корректность восстановления AES-ключа и содержимого контейнера.

3.5 Формирование и проверка подписи RSA/SHA-256

Подпись вычислялась по формуле $s = h(\text{file})^d \bmod n$, где $h(\text{file})$ - SHA-256 как целое число, приведенное по модулю n .

Формирование электронной подписи выполнялось путем вычисления SHA-256-хэш-образа сообщения, приведения полученного значения по модулю RSA и возведения результата в степень закрытого показателя. Проверка подписи выполнялась обратной операцией с использованием открытого показателя и сравнением восстановленного значения с повторно вычисленным хэш-образом сообщения. Последовательность операций формирования и проверки подписи представлена на рисунке 3.

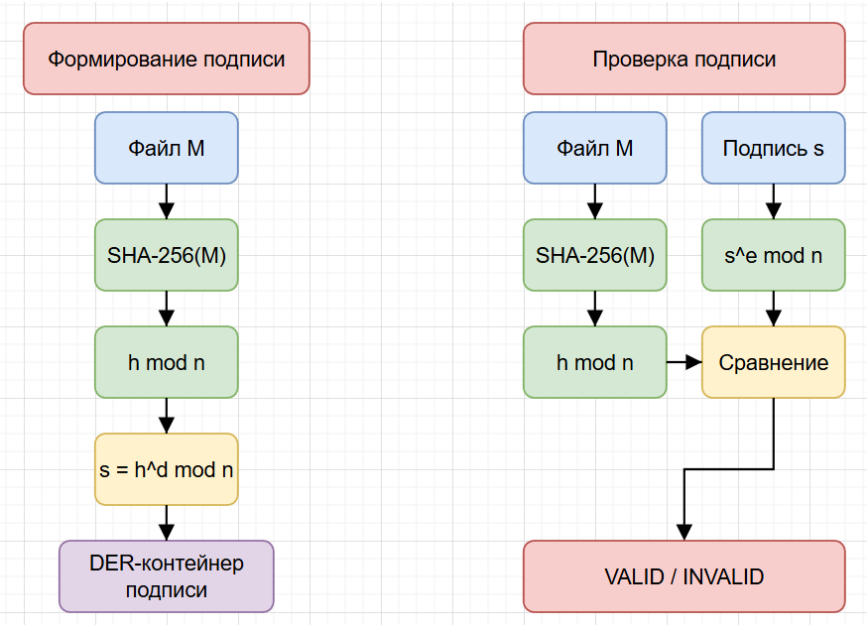


Рисунок 3 – Формирование и проверка электронной подписи RSA/SHA-256

Значение SHA-256-хэша сообщения и соответствующее значение по модулю n приведены в листинге 6:

Листинг 6 – SHA-256-хэш сообщения

```

SHA-256                                digest                                =
cbaebe27b3c1998bea7ad77752143802569363a436fc0c53288ee9d71946cc9e
h_sha256_mod_n                          =
92128252053848868482476689533025158553396365029028123219689047300
076057578654

```

Полученное целочисленное значение подписи RSA/SHA-256 приведено в листинге 7:

Листинг 7 – Значение подписи RSA/SHA-256

```
31219744972442003256613704681061508644683099076508176916509590362
73294800667316574993008764114444701298134324963166871032149471697
00391580141019448080721369370010494043377536045804026789837679636
44384681011321263111451974473106555215557779293416338054762100274
697712999652762898647223160698165652587492433911
```

Файл подписи хранится в ASN.1 DER-контейнере SignatureFile. Первые 100 байт DER-контейнера подписи RSA/SHA-256 приведены в листинге 8:

Листинг 8 – Первые 100 байт DER-контейнера подписи RSA/SHA-256

```
3081d70201010c0a7273612d7368613235360420cbaebe27b3c1998bea7ad7775
2143802569363a436fc0c53288ee9d71946cc9e022100cbaebe27b3c1998bea7a
d77752143802569363a436fc0c53288ee9d71946cc9e0281802c755937571b448
a603d
```

- проверка исходного файла: VALID;
- проверка измененного файла: INVALID.

При проверке подписи хэш файла вычисляется заново, затем проверяется равенство $\text{signature}^e \bmod n = \text{SHA-256}(\text{file}) \bmod n$. Значение hashValue из контейнера не принимается на доверие без пересчета по данным файла.

3.6 Дополнительное задание: RSA/CRC32 и подбор D3

Для демонстрации слабости CRC32 были подготовлены документы D1 и D2, после чего для D2 было рассчитано четырехбайтное дополнение patch. Значения контрольных сумм и параметры сформированного документа D3 приведены в таблице 2.

Таблица 2 – Значения CRC32 и параметры подбора D3

Параметр	Значение
CRC32(D1)	cdbee91e / 3451840798
CRC32(D2)	b444e08e / 3024412814
CRC32(D3)	cdbee91e / 3451840798

Таблица 2 – Значения CRC32 и параметры подбора D3 (продолжение)

Параметр	Значение
Размер D2	83 байт
Размер D3	87 байт
patch	ad67a5ae

Из таблицы 2 видно, что после добавления четырехбайтного дополнения значение CRC32 документа D3 совпало со значением CRC32 исходного документа D1, несмотря на то что D3 содержит изменённые данные D2.

Целочисленное значение подписи RSA/CRC32 для документа D1 приведено в листинге 9:

Листинг 9 – Значение подписи RSA/CRC32

```
32557292219350676996563547476030707533171689943996002297306911448
65109918306569836479960855766883728838299077530948253876991891311
39876839843835259144612014547293191542601295571438108014289254011
41553954871778193162843457939920852854312064911366556120407074890
289906816332177599960561263707935864694810632626
```

Файл подписи RSA/CRC32 также хранится в ASN.1 DER-контейнере SignatureFile. Первые 100 байт DER-контейнера подписи RSA/CRC32 приведены в листинге 10:

Листинг 10 – Первые 100 байт DER-контейнера подписи RSA/CRC32

```
30819e0201010c097273612d63726333320404cdbee91e020500cdbee91e02818
02e5cf5a2708b5b2ff0adf513c4923d5b1d6e20a780811353c40a2674f9f5aa15
f73ff2f9bddb6427592e502831d4870788733f7df3a2a8e3e421bebd6d1522a1c
31e00
```

- D3 начинается с полного содержимого D2: ОК;
- CRC32(D3) == CRC32(D1): ОК;
- D3 проходит проверку старой CRC32-подписью D1: VALID.

Файл D3 строится как $D3 = D2 \parallel \text{patch}$. Четыре байта patch рассчитаны по линейности CRC32 над GF(2), без перебора 2^{32} значений. Так как подпись RSA/CRC32 проверяет только CRC32, старая подпись D1 принимается и для D3.

Для демонстрации непригодности CRC32 в качестве криптографической хэш-функции была реализована процедура дополнения измененного документа D2. Программа вычисляет четыре корректирующих байта таким образом, чтобы итоговый документ D3 имел то же значение CRC32, что и исходный документ D1. В результате старая подпись ЭП-1, сформированная для D1, также принимается при проверке документа D3. Схема формирования документа D3 представлена на рисунке 4.

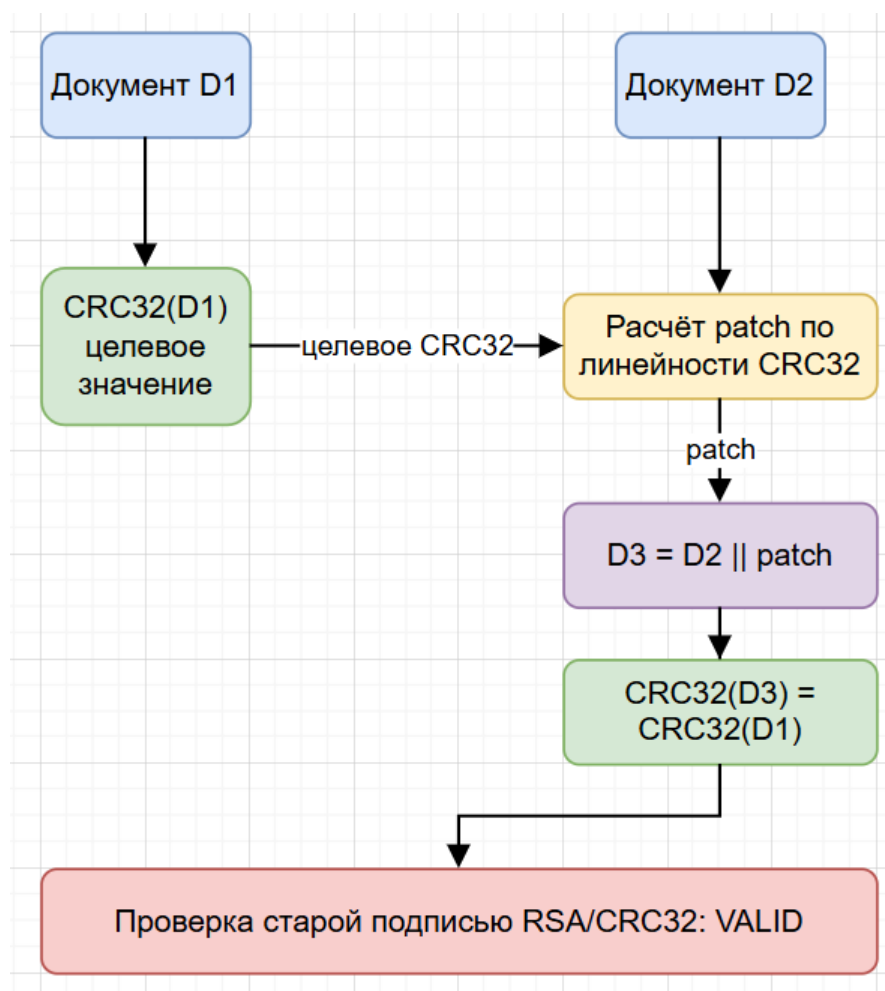


Рисунок 4 – Формирование документа D3 с сохранением CRC32

3.7 Итоговая проверка

Корректность реализации была подтверждена автоматическими тестами и демонстрационным запуском основных режимов программы. Были проверены генерация RSA-ключей, шифрование и расшифрование файла, формирование и проверка подписи RSA/SHA-256, обнаружение изменения подписанного файла, а также успешная демонстрация коллизии CRC32 для подписи RSA/CRC32. Итоговая схема проверки программы представлена на рисунке 5, а детализированные результаты выполнения отдельных проверок приведены в таблице 3.

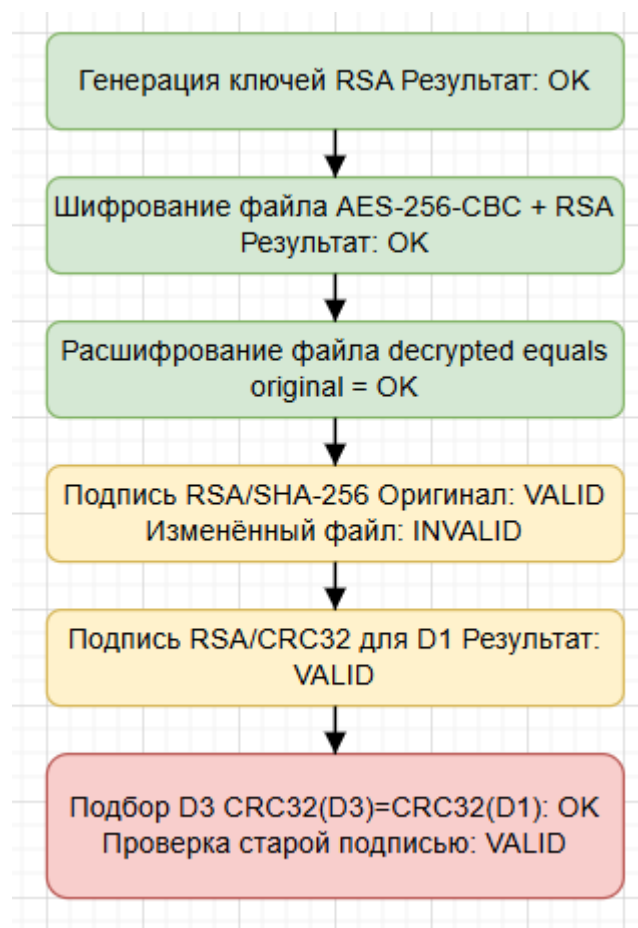


Рисунок 5 – Итоговая схема проверки реализованных функций

Таблица 3 – Итоговые результаты проверки

Проверка	Результат
Генерация ключей RSA	OK
Шифрование файла	OK
Расшифрование файла	OK
Расшифрованный файл совпадает с исходным	OK
RSA/SHA-256 для исходного файла	VALID
RSA/SHA-256 для измененного файла	INVALID
RSA/CRC32 для D1	VALID
$\text{CRC32}(D3) == \text{CRC32}(D1)$	OK
D3 проходит проверку старой CRC32-подписью D1	VALID

4 КОНТРОЛЬНЫЕ ВОПРОСЫ

4.1 Какие задачи положены в основу безопасности системы RSA?

Безопасность RSA опирается на вычислительную сложность нескольких математических задач. Основной задачей является разложение большого составного числа $n = p \cdot q$ на простые множители p и q . Если злоумышленник сможет разложить n , он сможет вычислить $\varphi(n) = (p - 1)(q - 1)$, а затем найти закрытый показатель d как обратный элемент к e по модулю $\varphi(n)$. Для схемы электронной подписи также важны свойства используемой хэш-функции. Подделка подписи не должна быть проще, чем разложение n на множители, обращение функции возведения в степень по модулю n или нахождение коллизий/прообразов хэш-функции. Поэтому в практических схемах RSA используют криптографические хэш-функции и стандартизованные схемы дополнения, например RSA-PSS для подписи и RSA-OAEP для шифрования.

4.2 Показать, что схема RSA работает корректно для любого сообщения $m \in \mathbb{Z}/n\mathbb{Z}$?

В RSA выбираются простые числа p и q , модуль $n = p \cdot q$, функция Эйлера $\varphi(n) = (p - 1)(q - 1)$, открытый показатель e и закрытый показатель d , для которых выполняется условие:

$$e \cdot d \equiv 1 \pmod{\varphi(n)}.$$

Отсюда существует целое k такое, что:

$$e \cdot d = 1 + k \cdot \varphi(n).$$

При шифровании сообщения m вычисляется:

$$c \equiv m^e \pmod{n}.$$

При расшифровании:

$$c^d \equiv (m^e)^d \equiv m^{(ed)} \equiv m^{(1+k\varphi(n))} \pmod{n}.$$

Если m взаимно просто с n , то по теореме Эйлера $m^{\varphi(n)} \equiv 1 \pmod{n}$.

Тогда:

$$m^{(1+k\varphi(n))} \equiv m \cdot (m^{\varphi(n)})^k \equiv m \pmod{n}.$$

Если m не взаимно просто с n , корректность всё равно сохраняется при рассмотрении сравнений по модулям p и q . По модулю p либо $m \equiv 0 \pmod{p}$, и тогда $m^{(ed)} \equiv 0 \equiv m \pmod{p}$, либо m не делится на p , и тогда по малой теореме Ферма $m^{(p-1)} \equiv 1 \pmod{p}$. Аналогично рассуждение выполняется по модулю q . Следовательно, $m^{(ed)} \equiv m$ одновременно по модулям p и q . Так как p и q взаимно просты, по китайской теореме об остатках получаем: $m^{(ed)} \equiv m \pmod{n}$.

Значит, расшифрование RSA корректно восстанавливает любое сообщение $m \in \mathbb{Z}/n\mathbb{Z}$.

4.3 Доказать, что задача разложения числа n на множители и задача вычисления функции Эйлера $\varphi(n)$ полиномиально эквивалентны.

Пусть известно разложение $n = p \cdot q$, где p и q — простые числа. Тогда значение функции Эйлера вычисляется напрямую:

$$\varphi(n) = (p - 1)(q - 1).$$

Это выполняется за полиномиальное время относительно длины n . Обратно, пусть известно n и $\varphi(n)$, но неизвестны p и q . Для $n = p \cdot q$ имеем:

$$\varphi(n) = (p - 1)(q - 1) = pq - p - q + 1 = n - (p + q) + 1.$$

Отсюда можно выразить сумму множителей:

$$p + q = n - \varphi(n) + 1.$$

Обозначим $S = p + q$. Тогда p и q являются корнями квадратного уравнения:

$$x^2 - Sx + n = 0.$$

Дискриминант этого уравнения:

$$D = S^2 - 4n = (p - q)^2.$$

Следовательно:

$$p = (S + \sqrt{D}) / 2,$$

$$q = (S - \sqrt{D}) / 2.$$

Таким образом, зная $\varphi(n)$, можно за полиномиальное время восстановить p и q . Следовательно, задача вычисления $\varphi(n)$ и задача факторизации n для RSA-модуля $n = p \cdot q$ полиномиально эквивалентны.

4.4 Показать, что схема RSA обладает свойством гомоморфности относительно операции умножения.

Пусть имеются два сообщения m_1 и m_2 . Их RSA-шифртексты при открытом ключе (n, e) равны:

$$c_1 \equiv m_1^e \bmod n,$$

$$c_2 \equiv m_2^e \bmod n.$$

Рассмотрим произведение шифртекстов:

$$c_1 \cdot c_2 \equiv m_1^e \cdot m_2^e \bmod n.$$

По свойствам степеней:

$$m_1^e \cdot m_2^e = (m_1 \cdot m_2)^e.$$

Следовательно:

$$c_1 \cdot c_2 \equiv (m_1 \cdot m_2)^e \bmod n.$$

Это означает, что произведение двух шифртекстов является корректным шифртекстом произведения соответствующих открытых сообщений. При расшифровании получим:

$$(c_1 \cdot c_2)^d \equiv m_1 \cdot m_2 \bmod n.$$

Таким образом, textbook RSA обладает мультипликативной гомоморфностью. Это свойство полезно для теоретического анализа, но в практической криптографии оно является опасным, поскольку позволяет выполнять управляемые преобразования шифртекстов. Поэтому на практике RSA применяют со схемами дополнения, например OAEP для шифрования и PSS для подписи.

5 ВЫВОДЫ

В работе реализована полная учебная схема защиты файла: RSA используется для защиты ключей и подписи, AES-256-CBC - для шифрования данных, DER - для структурированного хранения зашифрованного файла и подписи. Проверка RSA/SHA-256 корректно отвергает измененный файл. Демонстрация CRC32 показывает, что контрольные суммы нельзя использовать как криптографические хэши для электронной подписи.

Возможными уязвимостями учебной реализации являются отсутствие стандартизованного padding для textbook RSA, отсутствие защиты от атак по времени выполнения и хранение секретных параметров в демонстрационных JSON-файлах. В практических системах следует использовать проверенные криптографические библиотеки, схемы RSA-OAEP и RSA-PSS, постоянно обновляемые реализации и защищенное хранение закрытых ключей.

Полный исходный код разработанной программы приведён в приложениях 1–9.

ПРИЛОЖЕНИЕ 1. ЛИСТИНГ ФАЙЛА SRC/RSA.PY

```
"""Учебная реализация RSA.
```

```
Код намеренно оставлен явным для лабораторной работы. Здесь нет  
промышленных  
схем дополнения вроде OAEP/PSS, поэтому модуль не предназначен для  
реальной  
защиты данных.
```

```
"""
```

```
from __future__ import annotations
```

```
import json
```

```
import math
```

```
import secrets
```

```
from dataclasses import dataclass
```

```
from pathlib import Path
```

```
PUBLIC_EXPONENT = 65537
```

```
@dataclass(frozen=True)
```

```
class PublicKey:
```

```
    n: int
```

```
    e: int
```

```
@dataclass(frozen=True)
```

```
class PrivateKey:
```

```
    n: int
```

```
    e: int
```

```
    d: int
```

```
    p: int
```

```
    q: int
```

```
def _egcd(a: int, b: int) -> tuple[int, int, int]:
```

```
    # Расширенный алгоритм Евклида находит gcd(a, b) и коэффициенты  
x, y:
```

```
    #  $a \cdot x + b \cdot y = \text{gcd}(a, b)$ . Эти коэффициенты нужны для обратного  
элемента.
```

```
    if b == 0:
```

```
        return a, 1, 0
```

```
    g, x1, y1 = _egcd(b, a % b)
```

```

        return g, y1, x1 - (a // b) * y1

def mod_inverse(a: int, m: int) -> int:
    # Обратный элемент  $a^{-1} \bmod m$  существует только при  $\gcd(a, m) = 1$ .
    # В RSA это условие проверяется для  $e$  и  $\phi(n)$ .
    g, x, _ = _egcd(a, m)
    if g != 1:
        raise ValueError("inverse does not exist")
    return x % m

def _is_probable_prime(n: int, rounds: int = 32) -> bool:
    # Тест Миллера-Рабина: вероятностная проверка простоты больших чисел.
    # Для учебной генерации ключей он значительно быстрее полного перебора.
    if n < 2:
        return False
    small_primes = (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37)
    for prime in small_primes:
        if n == prime:
            return True
        if n % prime == 0:
            return False

    # Представляем  $n - 1$  как  $d * 2^s$ , где  $d$  нечетно.
    d = n - 1
    s = 0
    while d % 2 == 0:
        s += 1
        d //= 2

    for _ in range(rounds):
        a = secrets.randbelow(n - 3) + 2
        x = pow(a, d, n)
        if x in (1, n - 1):
            continue
        for _ in range(s - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True

```

```

def _generate_prime(bits: int) -> int:
    if bits < 16:
        raise ValueError("prime size is too small")
    while True:
        # Старший бит фиксирует требуемую длину, младший бит делает
        # число нечетным.
        candidate = secrets.randbits(bits)
        candidate |= (1 << (bits - 1)) | 1
        if _is_probable_prime(candidate):
            return candidate

def generate_keypair(bits: int = 1024) -> tuple[PrivateKey,
PublicKey]:
    if bits < 512:
        raise ValueError("RSA modulus must be at least 512 bits
for this lab")

    half = bits // 2
    while True:
        # Генерируем два независимых простых p и q, затем считаем
        # n и phi(n).
        p = _generate_prime(half)
        q = _generate_prime(bits - half)
        if p == q:
            continue
        phi = (p - 1) * (q - 1)
        if math.gcd(PUBLIC_EXPONENT, phi) != 1:
            continue
        n = p * q
        # Закрытая экспонента d является обратной к e по модулю
        # phi(n).
        d = mod_inverse(PUBLIC_EXPONENT, phi)
        private = PrivateKey(n=n, e=PUBLIC_EXPONENT, d=d, p=p,
q=q)
        public = PublicKey(n=n, e=PUBLIC_EXPONENT)
        return private, public

def encrypt_integer(message: int, public_key: PublicKey) -> int:
    # Базовая операция RSA без padding: c = m^e mod n.
    if not 0 <= message < public_key.n:
        raise ValueError("RSA message representative is out of
range")

```

```

        return pow(message, public_key.e, public_key.n)

def decrypt_integer(ciphertext: int, private_key: PrivateKey) ->
int:
    # Обратная операция RSA:  $m = c^d \bmod n$ .
    if not 0 <= ciphertext < private_key.n:
        raise ValueError("RSA ciphertext representative is out of
range")
    return pow(ciphertext, private_key.d, private_key.n)

def save_private_key(path: Path, key: PrivateKey) -> None:
    # JSON выбран как простой формат для отчета и ручной проверки
    параметров RSA.
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(
        json.dumps(key.__dict__, indent=2, sort_keys=True),
        encoding="utf-8",
    )

def save_public_key(path: Path, key: PublicKey) -> None:
    # Открытый ключ хранит только n и e, поэтому его можно
    передавать проверяющему.
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(
        json.dumps(key.__dict__, indent=2, sort_keys=True),
        encoding="utf-8",
    )

def load_private_key(path: Path) -> PrivateKey:
    # При загрузке явно приводим поля к int, чтобы не зависеть от
    деталей JSON-парсера.
    data = json.loads(path.read_text(encoding="utf-8"))
    return PrivateKey(
        n=int(data["n"]),
        e=int(data["e"]),
        d=int(data["d"]),
        p=int(data["p"]),
        q=int(data["q"]),
    )

def load_public_key(path: Path) -> PublicKey:

```

```
# Открытый ключ используется для шифрования AES-ключа и
# проверки подписей.
data = json.loads(path.read_text(encoding="utf-8"))
return PublicKey(n=int(data["n"]), e=int(data["e"]))

def int_to_fixed_bytes(value: int, length: int) -> bytes:
    return value.to_bytes(length, byteorder="big")

def int_from_bytes(data: bytes) -> int:
    return int.from_bytes(data, byteorder="big")
```

ПРИЛОЖЕНИЕ 2. ЛИСТИНГ ФАЙЛА SRC/AES.PY

```
"""Учебная реализация AES-256-CBC с дополнением PKCS#7.

Операции AES оставлены явно видимыми для отчета. Код не является
constant-time
и не должен использоваться как промышленная криптографическая
библиотека.
"""

from __future__ import annotations

# AES работает с блоком 128 бит: 16 байт или 4 слова по 32 бита.
BLOCK_SIZE = 16
NB = 4

# Для AES-256 ключ содержит 8 слов, а число раундов равно 14.
NK = 8
NR = 14

# S_BOX и INV_S_BOX реализуют нелинейную замену байтов SubBytes и
обратную замену.
S_BOX = [
    0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F, 0xC5, 0x30, 0x01,
    0x67, 0x2B, 0xFE, 0xD7, 0xAB, 0x76,
    0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD, 0xD4,
    0xA2, 0xAF, 0x9C, 0xA4, 0x72, 0xC0,
    0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC, 0x34, 0xA5,
    0xE5, 0xF1, 0x71, 0xD8, 0x31, 0x15,
    0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A, 0x07, 0x12,
    0x80, 0xE2, 0xEB, 0x27, 0xB2, 0x75,
    0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B,
    0xD6, 0xB3, 0x29, 0xE3, 0x2F, 0x84,
    0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB,
    0xBE, 0x39, 0x4A, 0x4C, 0x58, 0xCF,
    0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D, 0x33, 0x85, 0x45, 0xF9,
    0x02, 0x7F, 0x50, 0x3C, 0x9F, 0xA8,
    0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC, 0xB6,
    0xDA, 0x21, 0x10, 0xFF, 0xF3, 0xD2,
    0xCD, 0x0C, 0x13, 0xEC, 0x5F, 0x97, 0x44, 0x17, 0xC4, 0xA7,
    0x7E, 0x3D, 0x64, 0x5D, 0x19, 0x73,
    0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE,
    0xB8, 0x14, 0xDE, 0x5E, 0x0B, 0xDB,
    0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C, 0xC2, 0xD3,
    0xAC, 0x62, 0x91, 0x95, 0xE4, 0x79,
```

```
    0xE7, 0xC8, 0x37, 0x6D, 0x8D, 0xD5, 0x4E, 0xA9, 0x6C, 0x56,
    0xF4, 0xEA, 0x65, 0x7A, 0xAE, 0x08,
    0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD,
    0x74, 0x1F, 0x4B, 0xBD, 0x8B, 0x8A,
    0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E, 0x61, 0x35,
    0x57, 0xB9, 0x86, 0xC1, 0x1D, 0x9E,
    0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E,
    0x87, 0xE9, 0xCE, 0x55, 0x28, 0xDF,
    0x8C, 0xA1, 0x89, 0x0D, 0xBF, 0xE6, 0x42, 0x68, 0x41, 0x99,
    0x2D, 0x0F, 0xB0, 0x54, 0xBB, 0x16,
]
```

```
INV_S_BOX = [
    0x52, 0x09, 0x6A, 0xD5, 0x30, 0x36, 0xA5, 0x38, 0xBF, 0x40,
    0xA3, 0x9E, 0x81, 0xF3, 0xD7, 0xFB,
    0x7C, 0xE3, 0x39, 0x82, 0x9B, 0x2F, 0xFF, 0x87, 0x34, 0x8E,
    0x43, 0x44, 0xC4, 0xDE, 0xE9, 0xCB,
    0x54, 0x7B, 0x94, 0x32, 0xA6, 0xC2, 0x23, 0x3D, 0xEE, 0x4C,
    0x95, 0x0B, 0x42, 0xFA, 0xC3, 0x4E,
    0x08, 0x2E, 0xA1, 0x66, 0x28, 0xD9, 0x24, 0xB2, 0x76, 0x5B,
    0xA2, 0x49, 0x6D, 0x8B, 0xD1, 0x25,
    0x72, 0xF8, 0xF6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xD4, 0xA4,
    0x5C, 0xCC, 0x5D, 0x65, 0xB6, 0x92,
    0x6C, 0x70, 0x48, 0x50, 0xFD, 0xED, 0xB9, 0xDA, 0x5E, 0x15,
    0x46, 0x57, 0xA7, 0x8D, 0x9D, 0x84,
    0x90, 0xD8, 0xAB, 0x00, 0x8C, 0xBC, 0xD3, 0x0A, 0xF7, 0xE4,
    0x58, 0x05, 0xB8, 0xB3, 0x45, 0x06,
    0xD0, 0x2C, 0x1E, 0x8F, 0xCA, 0x3F, 0x0F, 0x02, 0xC1, 0xAF,
    0xBD, 0x03, 0x01, 0x13, 0x8A, 0x6B,
    0x3A, 0x91, 0x11, 0x41, 0x4F, 0x67, 0xDC, 0xEA, 0x97, 0xF2,
    0xCF, 0xCE, 0xF0, 0xB4, 0xE6, 0x73,
    0x96, 0xAC, 0x74, 0x22, 0xE7, 0xAD, 0x35, 0x85, 0xE2, 0xF9,
    0x37, 0xE8, 0x1C, 0x75, 0xDF, 0x6E,
    0x47, 0xF1, 0x1A, 0x71, 0x1D, 0x29, 0xC5, 0x89, 0x6F, 0xB7,
    0x62, 0x0E, 0xAA, 0x18, 0xBE, 0x1B,
    0xFC, 0x56, 0x3E, 0x4B, 0xC6, 0xD2, 0x79, 0x20, 0x9A, 0xDB,
    0xC0, 0xFE, 0x78, 0xCD, 0x5A, 0xF4,
    0x1F, 0xDD, 0xA8, 0x33, 0x88, 0x07, 0xC7, 0x31, 0xB1, 0x12,
    0x10, 0x59, 0x27, 0x80, 0xEC, 0x5F,
    0x60, 0x51, 0x7F, 0xA9, 0x19, 0xB5, 0x4A, 0x0D, 0x2D, 0xE5,
    0x7A, 0x9F, 0x93, 0xC9, 0x9C, 0xEF,
    0xA0, 0xE0, 0x3B, 0x4D, 0xAE, 0x2A, 0xF5, 0xB0, 0xC8, 0xEB,
    0xBB, 0x3C, 0x83, 0x53, 0x99, 0x61,
    0x17, 0x2B, 0x04, 0x7E, 0xBA, 0x77, 0xD6, 0x26, 0xE1, 0x69,
    0x14, 0x63, 0x55, 0x21, 0x0C, 0x7D,
]
```

```

RCON = [0x00, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1B,
0x36]

def _xor_bytes(left: bytes, right: bytes) -> bytes:
    return bytes(a ^ b for a, b in zip(left, right))

def _xtime(value: int) -> int:
    value <= 1
    if value & 0x100:
        value ^= 0x11B
    return value & 0xFF

def _mul(a: int, b: int) -> int:
    # Умножение байтов в поле GF(2^8), которое используется в
    MixColumns.
    result = 0
    for _ in range(8):
        if b & 1:
            result ^= a
        a = _xtime(a)
        b >>= 1
    return result

def _bytes_to_state(block: bytes) -> list[list[int]]:
    # AES хранит блок как матрицу 4x4 по столбцам: state[row][col].
    return [[block[row + 4 * col] for col in range(4)] for row in
range(4)]

def _state_to_bytes(state: list[list[int]]) -> bytes:
    return bytes(state[row][col] for col in range(4) for row in
range(4))

def _sub_word(word: list[int]) -> list[int]:
    return [S_BOX[b] for b in word]

def _rot_word(word: list[int]) -> list[int]:
    return word[1:] + word[:1]

```

```

def _expand_key(key: bytes) -> list[list[list[int]]]:
    # Key schedule разворачивает исходный 256-битный ключ в набор раундовых ключей.
    if len(key) != 32:
        raise ValueError("AES-256 key must be exactly 32 bytes")

    words = [list(key[i : i + 4]) for i in range(0, len(key), 4)]
    for i in range(NK, NB * (NR + 1)):
        temp = words[i - 1].copy()
        if i % NK == 0:
            temp = _sub_word(_rot_word(temp))
            temp[0] ^= RCON[i // NK]
        elif i % NK == 4:
            temp = _sub_word(temp)
        words.append([words[i - NK][j] ^ temp[j] for j in range(4)])

    round_keys = []
    for round_index in range(NR + 1):
        key_state = [[0] * 4 for _ in range(4)]
        for col in range(4):
            word = words[round_index * 4 + col]
            for row in range(4):
                key_state[row][col] = word[row]
        round_keys.append(key_state)
    return round_keys

def _add_round_key(state: list[list[int]], round_key: list[list[int]]) -> None:
    # AddRoundKey смешивает состояние с раундовым ключом через XOR.
    for row in range(4):
        for col in range(4):
            state[row][col] ^= round_key[row][col]

def _sub_bytes(state: list[list[int]]) -> None:
    # SubBytes заменяет каждый байт через S_BOX, добавляя нелинейность.
    for row in range(4):
        for col in range(4):
            state[row][col] = S_BOX[state[row][col]]

def _inv_sub_bytes(state: list[list[int]]) -> None:

```

```

# Обратная операция SubBytes для расшифрования.
for row in range(4):
    for col in range(4):
        state[row][col] = INV_S_BOX[state[row][col]]

def _shift_rows(state: list[list[int]]) -> None:
    # ShiftRows циклически сдвигает строки, распределяя байты по
    # столбцам.
    for row in range(1, 4):
        state[row] = state[row][row:] + state[row][:row]

def _inv_shift_rows(state: list[list[int]]) -> None:
    # Обратный сдвиг строк восстанавливает порядок перед обратным
    # SubBytes.
    for row in range(1, 4):
        state[row] = state[row][-row:] + state[row][:row]

def _mix_columns(state: list[list[int]]) -> None:
    # MixColumns перемешивает каждый столбец с помощью арифметики
    # GF(2^8).
    for col in range(4):
        a0, a1, a2, a3 = (state[row][col] for row in range(4))
        state[0][col] = _mul(a0, 2) ^ _mul(a1, 3) ^ a2 ^ a3
        state[1][col] = a0 ^ _mul(a1, 2) ^ _mul(a2, 3) ^ a3
        state[2][col] = a0 ^ a1 ^ _mul(a2, 2) ^ _mul(a3, 3)
        state[3][col] = _mul(a0, 3) ^ a1 ^ a2 ^ _mul(a3, 2)

def _inv_mix_columns(state: list[list[int]]) -> None:
    # Обратное перемешивание столбцов используется при
    # расшифровании блока.
    for col in range(4):
        a0, a1, a2, a3 = (state[row][col] for row in range(4))
        state[0][col] = _mul(a0, 14) ^ _mul(a1, 11) ^ _mul(a2, 13)
        ^ _mul(a3, 9)
        state[1][col] = _mul(a0, 9) ^ _mul(a1, 14) ^ _mul(a2, 11)
        ^ _mul(a3, 13)
        state[2][col] = _mul(a0, 13) ^ _mul(a1, 9) ^ _mul(a2, 14)
        ^ _mul(a3, 11)
        state[3][col] = _mul(a0, 11) ^ _mul(a1, 13) ^ _mul(a2, 9)
        ^ _mul(a3, 14)

def encrypt_block(block: bytes, key: bytes) -> bytes:

```

```

# Структура AES: начальный AddRoundKey, затем 13 полных раундов
и финальный
# раунд без MixColumns.
if len(block) != BLOCK_SIZE:
    raise ValueError("AES block must be exactly 16 bytes")
state = _bytes_to_state(block)
round_keys = _expand_key(key)
_add_round_key(state, round_keys[0])
for round_index in range(1, NR):
    _sub_bytes(state)
    _shift_rows(state)
    _mix_columns(state)
    _add_round_key(state, round_keys[round_index])
_sub_bytes(state)
_shift_rows(state)
_add_round_key(state, round_keys[NR])
return _state_to_bytes(state)

def decrypt_block(block: bytes, key: bytes) -> bytes:
    # Расшифрование применяет обратные преобразования в обратном
    порядке раундов.
    if len(block) != BLOCK_SIZE:
        raise ValueError("AES block must be exactly 16 bytes")
    state = _bytes_to_state(block)
    round_keys = _expand_key(key)
    _add_round_key(state, round_keys[NR])
    for round_index in range(NR - 1, 0, -1):
        _inv_shift_rows(state)
        _inv_sub_bytes(state)
        _add_round_key(state, round_keys[round_index])
        _inv_mix_columns(state)
    _inv_shift_rows(state)
    _inv_sub_bytes(state)
    _add_round_key(state, round_keys[0])
    return _state_to_bytes(state)

def pkcs7_pad(data: bytes) -> bytes:
    # PKCS#7 всегда добавляет от 1 до 16 байт, даже если данные
    кратны блоку.
    padding_len = BLOCK_SIZE - (len(data) % BLOCK_SIZE)
    return data + bytes([padding_len]) * padding_len

def pkcs7_unpad(data: bytes) -> bytes:
    # При снятии padding проверяем все байты дополнения, чтобы
    обнаружить порчу.

```

```

    if not data or len(data) % BLOCK_SIZE != 0:
        raise ValueError("invalid padded plaintext length")
    padding_len = data[-1]
    if padding_len < 1 or padding_len > BLOCK_SIZE:
        raise ValueError("invalid PKCS#7 padding")
    if data[-padding_len:] != bytes([padding_len]) * padding_len:
        raise ValueError("invalid PKCS#7 padding bytes")
    return data[:-padding_len]

def encrypt_cbc(plaintext: bytes, key: bytes, iv: bytes) -> bytes:
    # CBC: перед шифрованием каждый блок XOR-ится с предыдущим
    ciphertext.
    # Для первого блока используется случайный IV.
    if len(iv) != BLOCK_SIZE:
        raise ValueError("AES-CBC IV must be exactly 16 bytes")
    ciphertext = bytearray()
    previous = iv
    padded = pkcs7_pad(plaintext)
    for offset in range(0, len(padded), BLOCK_SIZE):
        block = padded[offset : offset + BLOCK_SIZE]
        encrypted = encrypt_block(_xor_bytes(block, previous),
key)
        ciphertext.extend(encrypted)
        previous = encrypted
    return bytes(ciphertext)

def decrypt_cbc(ciphertext: bytes, key: bytes, iv: bytes) -> bytes:
    # При CBC-расшифровании результат AES^-1 XOR-ится с предыдущим
    ciphertext.
    if len(iv) != BLOCK_SIZE:
        raise ValueError("AES-CBC IV must be exactly 16 bytes")
    if len(ciphertext) == 0 or len(ciphertext) % BLOCK_SIZE != 0:
        raise ValueError("AES-CBC ciphertext length must be a
positive block multiple")
    plaintext = bytearray()
    previous = iv
    for offset in range(0, len(ciphertext), BLOCK_SIZE):
        block = ciphertext[offset : offset + BLOCK_SIZE]
        plaintext_block = _xor_bytes(decrypt_block(block, key),
previous)
        plaintext.extend(plaintext_block)
        previous = block
    return pkcs7_unpad(bytes(plaintext))

```

ПРИЛОЖЕНИЕ 3. ЛИСТИНГ ФАЙЛА SRC/DER.PY

```
"""Минимальный DER-кодер/декодер для контейнера гибридного
шифрования."""

from __future__ import annotations

from dataclasses import dataclass

@dataclass(frozen=True)
class EncryptedFileContainer:
    """ASN.1 SEQUENCE с RSA-зашифрованным AES-ключом и AES-CBC
    ciphertext."""

    version: int
    encrypted_key: bytes
    iv: bytes
    ciphertext: bytes

def _encode_length(length: int) -> bytes:
    # DER использует TLV: Tag, Length, Value. Длина кодируется в
    короткой форме
    # для значений < 128 и в длинной форме для больших полей.
    if length < 0:
        raise ValueError("negative DER length")
    if length < 128:
        return bytes([length])
    raw = length.to_bytes((length.bit_length() + 7) // 8, "big")
    return bytes([0x80 | len(raw)]) + raw

def _encode_tlv(tag: int, value: bytes) -> bytes:
    return bytes([tag]) + _encode_length(len(value)) + value

def _encode_integer(value: int) -> bytes:
    # INTEGER в DER знаковый. Для положительных чисел с
    установленным старшим
    # битом добавляется ведущий 00, чтобы значение не трактовалось
    как отрицательное.
    if value < 0:
        raise ValueError("only non-negative integers are
supported")
```

```

raw      =      b"\x00"      if      value      ==      0      else
value.to_bytes((value.bit_length() + 7) // 8, "big")
    if raw[0] & 0x80:
        raw = b"\x00" + raw
    return _encode_tlv(0x02, raw)

def _encode_octet_string(value: bytes) -> bytes:
    # OCTET STRING хранит произвольные байты: RSA-блок, IV и
    ciphertext.
    return _encode_tlv(0x04, value)

def encode_container(container: EncryptedFileContainer) -> bytes:
    # EncryptedFile ::= SEQUENCE { version, encryptedKey, iv,
    ciphertext }.
    body = b"".join(
        (
            _encode_integer(container.version),
            _encode_octet_string(container.encrypted_key),
            _encode_octet_string(container.iv),
            _encode_octet_string(container.ciphertext),
        )
    )
    return _encode_tlv(0x30, body)

def _read_length(data: bytes, offset: int) -> tuple[int, int]:
    # При декодировании проверяем корректность DER-длины и
    запрещаем BER indefinite form.
    if offset >= len(data):
        raise ValueError("unexpected end of DER data")
    first = data[offset]
    offset += 1
    if first < 128:
        return first, offset
    count = first & 0x7F
    if count == 0:
        raise ValueError("indefinite DER lengths are not allowed")
    if offset + count > len(data):
        raise ValueError("truncated DER length")
    length = int.from_bytes(data[offset : offset + count], "big")
    if length < 128:
        raise ValueError("non-minimal DER length")
    return length, offset + count

```

```

def _read_tlv(data: bytes, offset: int, expected_tag: int) -> tuple[bytes, int]:
    # Каждое поле читается с ожидаемым tag, чтобы контейнер имел заданную структуру.
    if offset >= len(data) or data[offset] != expected_tag:
        raise ValueError(f"expected DER tag 0x{expected_tag:02x}")
    length, value_offset = _read_length(data, offset + 1)
    end = value_offset + length
    if end > len(data):
        raise ValueError("truncated DER value")
    return data[value_offset:end], end

def _decode_integer(value: bytes) -> int:
    if not value:
        raise ValueError("empty DER integer")
    return int.from_bytes(value, "big")

def decode_container(data: bytes) -> EncryptedFileContainer:
    # Декодер проверяет отсутствие лишних байтов снаружи и внутри SEQUENCE.
    body, offset = _read_tlv(data, 0, 0x30)
    if offset != len(data):
        raise ValueError("trailing data after DER sequence")

    inner = 0
    version_raw, inner = _read_tlv(body, inner, 0x02)
    encrypted_key, inner = _read_tlv(body, inner, 0x04)
    iv, inner = _read_tlv(body, inner, 0x04)
    ciphertext, inner = _read_tlv(body, inner, 0x04)
    if inner != len(body):
        raise ValueError("trailing data inside DER sequence")

    return EncryptedFileContainer(
        version=_decode_integer(version_raw),
        encrypted_key=encrypted_key,
        iv=iv,
        ciphertext=ciphertext,
    )

```

ПРИЛОЖЕНИЕ 4. ЛИСТИНГ ФАЙЛА SRC/SIGDER.PY

```
"""ASN.1 DER-контейнеры для файлов RSA-подписей."""

from __future__ import annotations

from dataclasses import dataclass

SIGNATURE_VERSION = 1

@dataclass(frozen=True)
class SignatureContainer:
    """Структура SignatureFile.

    algorithm указывает вариант подписи, hash_value хранит
    исходный digest,
    hash_mod_n хранит digest как число после приведения по модулю
    n, а signature
    содержит RSA-значение  $s = h^d \bmod n$ .
    """

    version: int
    algorithm: str
    hash_value: bytes
    hash_mod_n: int
    signature: int

def _encode_length(length: int) -> bytes:
    # DER кодирует каждое поле как TLV. Длина может быть короткой
    или длинной.
    if length < 0:
        raise ValueError("negative DER length")
    if length < 128:
        return bytes([length])
    raw = length.to_bytes((length.bit_length() + 7) // 8, "big")
    return bytes([0x80 | len(raw)]) + raw

def _encode_tlv(tag: int, value: bytes) -> bytes:
    return bytes([tag]) + _encode_length(len(value)) + value

def _encode_integer(value: int) -> bytes:
```

```

# DER INTEGER знаковый, поэтому для положительного числа иногда
нужен ведущий 00.
    if value < 0:
        raise ValueError("only non-negative integers are
supported")
    raw = b"\x00" if value == 0 else
value.to_bytes((value.bit_length() + 7) // 8, "big")
    if raw[0] & 0x80:
        raw = b"\x00" + raw
    return _encode_tlv(0x02, raw)

def _encode_utf8_string(value: str) -> bytes:
    # algorithm хранится как UTF8String: "rsa-sha256" или "rsa-
crc32".
    return _encode_tlv(0x0C, value.encode("utf-8"))

def _encode_octet_string(value: bytes) -> bytes:
    return _encode_tlv(0x04, value)

def encode_signature_container(container: SignatureContainer) ->
bytes:
    # SignatureFile ::= SEQUENCE { version, algorithm, hashValue,
hashModN, signature }.
    body = b"".join(
        (
            _encode_integer(container.version),
            _encode_utf8_string(container.algorithm),
            _encode_octet_string(container.hash_value),
            _encode_integer(container.hash_mod_n),
            _encode_integer(container.signature),
        )
    )
    return _encode_tlv(0x30, body)

def _read_length(data: bytes, offset: int) -> tuple[int, int]:
    # Декодер принимает только DER-форму: без неопределенной длины
и без
    # неминимального long-form кодирования.
    if offset >= len(data):
        raise ValueError("unexpected end of DER data")
    first = data[offset]
    offset += 1

```

```

    if first < 128:
        return first, offset
    count = first & 0x7F
    if count == 0:
        raise ValueError("indefinite DER lengths are not allowed")
    if offset + count > len(data):
        raise ValueError("truncated DER length")
    length = int.from_bytes(data[offset : offset + count], "big")
    if length < 128:
        raise ValueError("non-minimal DER length")
    return length, offset + count

def _read_tlv(data: bytes, offset: int, expected_tag: int) ->
tuple[bytes, int]:
    if offset >= len(data) or data[offset] != expected_tag:
        raise ValueError(f"expected DER tag 0x{expected_tag:02x}")
    length, value_offset = _read_length(data, offset + 1)
    end = value_offset + length
    if end > len(data):
        raise ValueError("truncated DER value")
    return data[value_offset:end], end

def _decode_integer(value: bytes) -> int:
    # Отрицательные и неминимально закодированные INTEGER
отклоняются, чтобы
    # одна и та же подпись не имела нескольких допустимых бинарных
представлений.
    if not value:
        raise ValueError("empty DER integer")
    if len(value) > 1 and value[0] == 0 and not (value[1] & 0x80):
        raise ValueError("non-minimal DER integer")
    if value[0] & 0x80:
        raise ValueError("negative DER integer is not supported")
    return int.from_bytes(value, "big")

def decode_signature_container(data: bytes) -> SignatureContainer:
    # При чтении проверяем точный порядок полей и отсутствие
хвостовых байтов.
    body, offset = _read_tlv(data, 0, 0x30)
    if offset != len(data):
        raise ValueError("trailing data after DER sequence")

    inner = 0

```

```
version_raw, inner = _read_tlv(body, inner, 0x02)
algorithm_raw, inner = _read_tlv(body, inner, 0x0C)
hash_value, inner = _read_tlv(body, inner, 0x04)
hash_mod_n_raw, inner = _read_tlv(body, inner, 0x02)
signature_raw, inner = _read_tlv(body, inner, 0x02)
if inner != len(body):
    raise ValueError("trailing data inside DER sequence")

return SignatureContainer(
    version=_decode_integer(version_raw),
    algorithm=algorithm_raw.decode("utf-8"),
    hash_value=hash_value,
    hash_mod_n=_decode_integer(hash_mod_n_raw),
    signature=_decode_integer(signature_raw),
)
```

ПРИЛОЖЕНИЕ 5. ЛИСТИНГ ФАЙЛА SRC/HYBRID.PY

```
"""Гибридное шифрование файла: AES-256-CBC плюс RSA-обертка
ключа."""

from __future__ import annotations

import json
import secrets
from pathlib import Path

from src.aes import BLOCK_SIZE, decrypt_cbc, encrypt_cbc
from src.der import EncryptedFileContainer, decode_container,
encode_container
from src.rsa import (
    PrivateKey,
    PublicKey,
    decrypt_integer,
    encrypt_integer,
    int_from_bytes,
    int_to_fixed_bytes,
)

AES_KEY_SIZE = 32
CONTAINER_VERSION = 1

def encrypt_file(
    input_path: Path,
    public_key: PublicKey,
    output_path: Path,
    debug_json_path: Path | None = None,
) -> None:
    plaintext = input_path.read_bytes()
    # Для каждого шифрования создаются новый AES-256 ключ и новый
    IV.
    aes_key = secrets.token_bytes(AES_KEY_SIZE)
    iv = secrets.token_bytes(BLOCK_SIZE)

    # RSA шифрует только короткий AES-ключ, а не весь файл.
    encrypted_key_int = encrypt_integer(int_from_bytes(aes_key),
public_key)
    rsa_size = (public_key.n.bit_length() + 7) // 8
    encrypted_key = int_to_fixed_bytes(encrypted_key_int,
rsa_size)
```

```

# Содержимое файла шифруется блочным режимом AES-CBC с PKCS#7
padding.
ciphertext = encrypt_cbc(plaintext, aes_key, iv)
container = EncryptedFileContainer(
    version=CONTAINER_VERSION,
    encrypted_key=encrypted_key,
    iv=iv,
    ciphertext=ciphertext,
)
# Все поля складываются в структурированный ASN.1 DER-
контейнер.
container_der = encode_container(container)
output_path.parent.mkdir(parents=True, exist_ok=True)
output_path.write_bytes(container_der)

if debug_json_path is not None:
    # Debug JSON нужен только для отчета: он показывает
    # ключевые промежуточные значения в hex и не должен использоваться как защищенный
    # формат хранения.
    debug_json_path.parent.mkdir(parents=True, exist_ok=True)
    debug_json_path.write_text(
        json.dumps(
            {
                "aes_key_hex": aes_key.hex(),
                "iv_hex": iv.hex(),
                "encrypted_key_hex": encrypted_key.hex(),
                "ciphertext_first_100_hex":
ciphertext[:100].hex(),
                "container_first_100_hex":
container_der[:100].hex(),
            },
            indent=2,
        ),
        encoding="utf-8",
    )

def decrypt_file(input_path: Path, private_key: PrivateKey,
output_path: Path) -> None:
    # Расшифрование начинается с разбора DER-контейнера и проверки
    # версии.
    container = decode_container(input_path.read_bytes())
    if container.version != CONTAINER_VERSION:
        raise ValueError(f"unsupported container version:
{container.version}")

```

```
if len(container.iv) != BLOCK_SIZE:
    raise ValueError("invalid IV length in container")

encrypted_key_int = int_from_bytes(container.encrypted_key)
# Замкнутый RSA-ключ восстанавливает AES-ключ, после чего AES-
CBC возвращает plaintext.
aes_key_int = decrypt_integer(encrypted_key_int, private_key)
aes_key = int_to_fixed_bytes(aes_key_int, AES_KEY_SIZE)
plaintext = decrypt_cbc(container.ciphertext, aes_key,
container.iv)

output_path.parent.mkdir(parents=True, exist_ok=True)
output_path.write_bytes(plaintext)
```

ПРИЛОЖЕНИЕ 6. ЛИСТИНГ ФАЙЛА SRC/SIGNATURES.PY

```
"""Учебные RSA-подписи для SHA-256 и CRC32."""

from __future__ import annotations

import binascii
import hashlib
from pathlib import Path

from src.rsa import PrivateKey, PublicKey
from src.sigder import (
    SIGNATURE_VERSION,
    SignatureContainer,
    decode_signature_container,
    encode_signature_container,
)

SHA256_ALGORITHM = "rsa-sha256"
CRC32_ALGORITHM = "rsa-crc32"

def sha256_hash_int(data: bytes, modulus: int) -> int:
    # SHA-256 дает 32-байтный digest, который интерпретируется как
    # большое число.
    digest = hashlib.sha256(data).digest()
    return int.from_bytes(digest, byteorder="big") % modulus

def crc32_hash_int(data: bytes, modulus: int) -> int:
    # CRC32 оставлен только для демонстрации уязвимости из
    # дополнительного задания.
    # Это не криптографический hash.
    return (binascii.crc32(data) & 0xFFFFFFFF) % modulus

def sha256_digest(data: bytes) -> bytes:
    return hashlib.sha256(data).digest()

def crc32_digest(data: bytes) -> bytes:
    return (binascii.crc32(data) & 0xFFFFFFFF).to_bytes(4,
byteorder="big")
```

```

def sign_sha256(data: bytes, private_key: PrivateKey) -> int:
    # Формула подписи RSA:  $s = h^d \bmod n$ .
    h = sha256_hash_int(data, private_key.n)
    return pow(h, private_key.d, private_key.n)

def verify_sha256(data: bytes, signature: int, public_key:
PublicKey) -> bool:
    # Проверка RSA:  $s^e \bmod n$  должно совпасть с  $h(\text{file}) \bmod n$ .
    h = sha256_hash_int(data, public_key.n)
    return pow(signature, public_key.e, public_key.n) == h

def sign_crc32(data: bytes, private_key: PrivateKey) -> int:
    # Та же RSA-формула применяется к CRC32-значению для учебной
атаки.
    h = crc32_hash_int(data, private_key.n)
    return pow(h, private_key.d, private_key.n)

def verify_crc32(data: bytes, signature: int, public_key:
PublicKey) -> bool:
    h = crc32_hash_int(data, public_key.n)
    return pow(signature, public_key.e, public_key.n) == h

def create_sha256_container(data: bytes, private_key: PrivateKey)
-> SignatureContainer:
    # В контейнер записываем и исходный digest, и digest после
приведения по модулю n.
    digest = sha256_digest(data)
    hash_mod_n = int.from_bytes(digest, byteorder="big") %
private_key.n
    return SignatureContainer(
        version=SIGNATURE_VERSION,
        algorithm=SHA256_ALGORITHM,
        hash_value=digest,
        hash_mod_n=hash_mod_n,
        signature=pow(hash_mod_n, private_key.d, private_key.n),
    )

def create_crc32_container(data: bytes, private_key: PrivateKey) -
> SignatureContainer:
    digest = crc32_digest(data)

```

```

    hash_mod_n = int.from_bytes(digest, byteorder="big") %
private_key.n
    return SignatureContainer(
        version=SIGNATURE_VERSION,
        algorithm=CRC32_ALGORITHM,
        hash_value=digest,
        hash_mod_n=hash_mod_n,
        signature=pow(hash_mod_n, private_key.d, private_key.n),
    )

def save_signature_container(path: Path, container:
SignatureContainer) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_bytes(encode_signature_container(container))

def load_signature_container(path: Path) -> SignatureContainer:
    return decode_signature_container(path.read_bytes())

def verify_sha256_container(data: bytes, container:
SignatureContainer, public_key: PublicKey) -> bool:
    # hashValue и hashModN из файла подписи не доверяются сами по
себе:
    # проверка заново считает digest по данным файла и сравнивает
все поля.
    digest = sha256_digest(data)
    hash_mod_n = int.from_bytes(digest, byteorder="big") %
public_key.n
    return (
        container.version == SIGNATURE_VERSION
        and container.algorithm == SHA256_ALGORITHM
        and container.hash_value == digest
        and container.hash_mod_n == hash_mod_n
        and verify_sha256(data, container.signature, public_key)
    )

def verify_crc32_container(data: bytes, container:
SignatureContainer, public_key: PublicKey) -> bool:
    # Для CRC32 также заново считаем значение по файлу, иначе
контейнер можно было бы
    # подменить согласованными hashValue/hashModN без связи с
проверяемыми данными.
    digest = crc32_digest(data)

```

```

    hash_mod_n = int.from_bytes(digest, byteorder="big") %
public_key.n
    return (
        container.version == SIGNATURE_VERSION
        and container.algorithm == CRC32_ALGORITHM
        and container.hash_value == digest
        and container.hash_mod_n == hash_mod_n
        and verify_crc32(data, container.signature, public_key)
    )

def save_signature(path: Path, container: SignatureContainer) ->
None:
    """Backward-compatible name for saving DER signature
containers."""
    save_signature_container(path, container)

def load_signature(path: Path) -> SignatureContainer:
    """Backward-compatible name for loading DER signature
containers."""
    return load_signature_container(path)

```

ПРИЛОЖЕНИЕ 7. ЛИСТИНГ ФАЙЛА SRC/CRCFORGE.PY

```
"""Подбор четырехбайтного суффикса для CRC32.

CRC32 линейна над GF(2). Вместо перебора всех 2^32 суффиксов
строится система
из 32 столбцов, описывающая влияние каждого бита patch на итоговую
CRC32.
"""

from __future__ import annotations

import binascii
from pathlib import Path

def crc32_bytes(data: bytes) -> int:
    return binascii.crc32(data) & 0xFFFFFFFF

def _solve_gf2(columns: list[int], target: int) -> int:
    # В младших 32 битах строки хранятся коэффициенты, в бите 32 -
    # правая часть.
    rows = []
    for bit in range(32):
        # Каждая строка системы отвечает за один бит итоговой
        CRC32.
        coeffs = 0
        for column_index, column in enumerate(columns):
            if (column >> bit) & 1:
                coeffs |= 1 << column_index
        rows.append(coeffs | (((target >> bit) & 1) << 32))

    pivot_row = 0
    for col in range(32):
        # Метод Гаусса над GF(2): сложение строк заменяется XOR.
        selected = next((r for r in range(pivot_row, 32) if
        (rows[r] >> col) & 1), None)
        if selected is None:
            continue
        rows[pivot_row], rows[selected] = rows[selected],
        rows[pivot_row]
        for r in range(32):
            if r != pivot_row and ((rows[r] >> col) & 1):
                rows[r] ^= rows[pivot_row]
        pivot_row += 1

    for row in rows:
```

```

        if (row & 0xFFFFFFFF) == 0 and ((row >> 32) & 1):
            raise ValueError("CRC32 patch system has no solution")

    solution = 0
    for row in rows:
        coeffs = row & 0xFFFFFFFF
        if coeffs and coeffs & (coeffs - 1) == 0 and ((row >> 32)
& 1):
            solution |= coeffs
    return solution

def crc32_patch_for_append(prefix: bytes, target_crc: int) ->
bytes:
    # Четыре байта дают 32 управляемых бита, чего достаточно для
управления
    # 32-битным состоянием CRC32 без полного перебора.
    zero_patch = b"\x00\x00\x00\x00"
    base_crc = crc32_bytes(prefix + zero_patch)
    columns = []
    for bit in range(32):
        # Столбец показывает, как один установленный бит patch
меняет CRC.
        patch_value = 1 << bit
        patch = patch_value.to_bytes(4, byteorder="little")
        columns.append(crc32_bytes(prefix + patch) ^ base_crc)

    solution = _solve_gf2(columns, target_crc ^ base_crc)
    patch = solution.to_bytes(4, byteorder="little")
    if crc32_bytes(prefix + patch) != target_crc:
        raise ValueError("internal CRC32 patch verification
failed")
    return patch

def forge_crc32_file(original_path: Path, modified_path: Path,
output_path: Path) -> None:
    # Итоговый файл строится как D3 = D2 || patch: содержимое D2
сохраняется полностью.
    original = original_path.read_bytes()
    modified = modified_path.read_bytes()
    patch = crc32_patch_for_append(modified,
crc32_bytes(original))
    output_path.parent.mkdir(parents=True, exist_ok=True)
    output_path.write_bytes(modified + patch)

```

ПРИЛОЖЕНИЕ 8. ЛИСТИНГ ФАЙЛА SRC/MAIN.PY

```
"""CLI-интерфейс для команд KMZI Lab1."""

from __future__ import annotations
import argparse
import sys
from pathlib import Path
from src.crcforge import forge_crc32_file
from src.hybrid import decrypt_file, encrypt_file
from src.rsa import generate_keypair, load_private_key,
load_public_key, save_private_key, save_public_key
from src.signatures import (
    create_crc32_container,
    create_sha256_container,
    load_signature,
    save_signature,
    verify_crc32_container,
    verify_sha256_container,
)

def build_parser() -> argparse.ArgumentParser:
    # Все лабораторные действия оформлены как subcommands, чтобы
    имена команд
    # оставались стабильными для демонстрации и отчета.
    parser = argparse.ArgumentParser(prog="python -m src.main",
description="KMZI Lab1 RSA/AES tools")
    subparsers = parser.add_subparsers(dest="command",
required=True)

    keygen = subparsers.add_parser("keygen", help="Generate RSA
key pair")
    keygen.add_argument("--bits", type=int, default=1024)
    keygen.add_argument("--out", type=Path, required=True)

    encrypt = subparsers.add_parser("encrypt", help="Encrypt file
into ASN.1 DER container")
    encrypt.add_argument("--file", type=Path, required=True)
    encrypt.add_argument("--public-key", type=Path,
required=True)
    encrypt.add_argument("--out", type=Path, required=True)
    encrypt.add_argument("--debug-json", type=Path, help="Write
educational encryption debug report")

    decrypt = subparsers.add_parser("decrypt", help="Decrypt ASN.1
DER container")
```

```

    decrypt.add_argument("--file", type=Path, required=True)
    decrypt.add_argument("--private-key", type=Path,
required=True)
    decrypt.add_argument("--out", type=Path, required=True)

    sign_sha = subparsers.add_parser("sign-sha256", help="Sign
file with RSA/SHA-256")
    sign_sha.add_argument("--file", type=Path, required=True)
    sign_sha.add_argument("--private-key", type=Path,
required=True)
    sign_sha.add_argument("--signature", type=Path,
required=True)

    verify_sha = subparsers.add_parser("verify-sha256",
help="Verify RSA/SHA-256 signature")
    verify_sha.add_argument("--file", type=Path, required=True)
    verify_sha.add_argument("--public-key", type=Path,
required=True)
    verify_sha.add_argument("--signature", type=Path,
required=True)

    sign_crc = subparsers.add_parser("sign-crc32", help="Sign file
with RSA/CRC32")
    sign_crc.add_argument("--file", type=Path, required=True)
    sign_crc.add_argument("--private-key", type=Path,
required=True)
    sign_crc.add_argument("--signature", type=Path,
required=True)

    verify_crc = subparsers.add_parser("verify-crc32",
help="Verify RSA/CRC32 signature")
    verify_crc.add_argument("--file", type=Path, required=True)
    verify_crc.add_argument("--public-key", type=Path,
required=True)
    verify_crc.add_argument("--signature", type=Path,
required=True)

    forge = subparsers.add_parser("forge-crc32", help="Create D3 =
D2 || patch with CRC32 (D3)=CRC32 (D1)")
    forge.add_argument("--original", type=Path, required=True)
    forge.add_argument("--modified", type=Path, required=True)
    forge.add_argument("--out", type=Path, required=True)

    return parser

def main(argv: list[str] | None = None) -> int:

```

```

parser = build_parser()
args = parser.parse_args(argv)

try:
    if args.command == "keygen":
        # keygen создает пару RSA-ключей и сохраняет их в JSON
        для отчета.
        private, public = generate_keypair(args.bits)
        save_private_key(args.out / "private.json", private)
        save_public_key(args.out / "public.json", public)
        print(f"Keys saved to: {args.out}")
        return 0

    if args.command == "encrypt":
        # encrypt формирует ASN.1 DER-контейнер с RSA-wrapped
        AES key и ciphertext.
        encrypt_file(args.file,
load_public_key(args.public_key), args.out, args.debug_json)
        print(f"Encrypted file saved to: {args.out}")
        if args.debug_json:
            print(f"Debug JSON saved to: {args.debug_json}")
        return 0

    if args.command == "decrypt":
        # decrypt разбирает DER-контейнер и восстанавливает
        исходный файл.
        decrypt_file(args.file,
load_private_key(args.private_key), args.out)
        print(f"Decrypted file saved to: {args.out}")
        return 0

    if args.command == "sign-sha256":
        # Подпись сохраняется не как текстовое число, а как
        DER SignatureFile.
        container =
create_sha256_container(args.file.read_bytes(),
load_private_key(args.private_key))
        save_signature(args.signature, container)
        print(f"Signature saved to: {args.signature}")
        return 0

    if args.command == "verify-sha256":
        # INVALID возвращает код 1, чтобы CLI можно было
        использовать в скриптах.
        ok = verify_sha256_container(
            args.file.read_bytes(),

```

```

        load_signature(args.signature),
        load_public_key(args.public_key),
    )
    print("VALID" if ok else "INVALID")
    return 0 if ok else 1

    if args.command == "sign-crc32":
        # CRC32-подпись нужна для демонстрации дополнительного
задания.
        container =
create_crc32_container(args.file.read_bytes(),
load_private_key(args.private_key))
        save_signature(args.signature, container)
        print(f"Signature saved to: {args.signature}")
        return 0
    if args.command == "verify-crc32":
        # Проверка CRC32 использует тот же DER-контейнер, но
другой algorithm id.
        ok = verify_crc32_container(
            args.file.read_bytes(),
            load_signature(args.signature),
            load_public_key(args.public_key),
        )
        print("VALID" if ok else "INVALID")
        return 0 if ok else 1

    if args.command == "forge-crc32":
        # forge-crc32 строит D3 = D2 || patch для совпадения
CRC32 с D1.
        forge_crc32_file(args.original, args.modified,
args.out)
        print(f"Forged file saved to: {args.out}")
        return 0

    except (OSError, ValueError) as exc:
        # Верхний уровень CLI превращает ошибки файлов/формата в
понятный код 1.
        print(f"ERROR: {exc}")
        return 1

    parser.error("unknown command")
    return 2

if __name__ == "__main__":
    raise SystemExit(main(sys.argv[1:]))

```

ПРИЛОЖЕНИЕ 9. ЛИСТИНГ ФАЙЛА SCRIPTS/GENERATE_REPORT_MATERIALS.PY

```
"""Генерация воспроизводимых материалов для отчета по KMZI Lab1.

Скрипт использует только существующую реализацию Lab1 и стандартную
библиотеку.
Случайные RSA/AES значения могут меняться при каждом запуске, но
структура
выходных Markdown/JSON/TXT файлов остается одинаковой.
"""

from __future__ import annotations

import hashlib
import json
import shutil
import sys
from pathlib import Path
from typing import Any

SCRIPT_PATH = Path(__file__).resolve()
DEFAULT_LAB_ROOT = SCRIPT_PATH.parents[1]
if str(DEFAULT_LAB_ROOT) not in sys.path:
    sys.path.insert(0, str(DEFAULT_LAB_ROOT))

from src.crcforge import crc32_bytes, forge_crc32_file
from src.hybrid import decrypt_file, encrypt_file
from src.rsa import generate_keypair, save_private_key,
save_public_key
from src.signatures import (
    create_crc32_container,
    create_sha256_container,
    load_signature_container,
    save_signature,
    verify_crc32_container,
    verify_sha256_container,
)

REPORT_FILES = (
    "report_data.md",
    "report_data.json",
    "verification_log.txt",
    "hex_dump.md",
```

```

        "crc32_demo.md",
        "theory.md",
        "control_questions.md",
        "stand_commands.md",
        "full_report_draft.md",
    )

def _clean_dir(path: Path) -> None:
    # artifacts/report_run хранит временные ключи, подписи и
    # шифртексты одного запуска.
    if path.exists():
        shutil.rmtree(path)
    path.mkdir(parents=True, exist_ok=True)

def _hex_block(value: str, width: int = 64) -> str:
    return "\n".join(value[i : i + width] for i in range(0,
len(value), width))

def _code_block(value: int | str) -> str:
    return f"```text\n{value}\n```"

def _status(value: bool) -> str:
    return "VALID" if value else "INVALID"

def _write_json(path: Path, data: dict[str, Any]) -> None:
    path.write_text(json.dumps(data,
                                indent=2,
                                ensure_ascii=False), encoding="utf-8")

def _build_report_markdown(data: dict[str, Any]) -> str:
    rsa = data["rsa"]
    aes = data["aes"]
    files = data["files"]
    sha = data["sha256_signature"]
    crc = data["crc32_signature"]

    return f"""# Данные для отчета по Lab1

## 1. Параметры RSA

Размер модуля RSA: **{rsa["modulus_bit_length"]} бит**.
```

```
**p:**  
{_code_block(rsa["p"])}  

```

```
**q:**  
{_code_block(rsa["q"])}  

```

```
**n = p * q:**  
{_code_block(rsa["n"])}  

```

```
**e:**  
{_code_block(rsa["e"])}  

```

```
**d:**  
{_code_block(rsa["d"])}  

```

2. Гибридное шифрование RSA + AES-256-CBC

Исходный файл `data/message.txt` был зашифрован по гибридной схеме: содержимое файла шифруется AES-256-CBC, а 32-байтный AES-ключ шифруется RSA.

```
**AES-256 ключ, hex:**  
```text  
{_hex_block(aes["aes_key_hex"])}
```
```

```
**IV, hex:**  
```text  
{aes["iv_hex"]}
```
```

```
**Зашифрованный AES-ключ, hex:**  
```text  
{_hex_block(aes["encrypted_key_hex"])}
```
```

Размер исходного файла: **{files["original_message_size"]}** байт**.**
Размер DER-контейнера: **{files["encrypted_container_size"]}** байт**.**

```
**Первые 100 байт DER-контейнера, hex:**  
```text  
{_hex_block(files["container_first_100_hex"])}
```
```

```
**Первые 100 байт AES ciphertext, hex:**  
```text  
{_hex_block(files["ciphertext_first_100_hex"])}
```
```

3. Расшифрование

- DER-контейнер разобран: **ОК**
- AES-ключ расшифрован закрытым RSA-ключом: **ОК**
- Содержимое файла восстановлено AES-256-CBC: **ОК**

Результат сравнения расшифрованного файла с исходным: **{"OK" if files["decrypted_equals_original"] else "FAIL"}**.

4. Электронная подпись RSA/SHA-256

Подпись вычислялась по формуле $s = h(\text{file})^d \bmod n$, где $h(\text{file})$ - SHA-256 как целое число, приведенное по модулю n .

```
**SHA-256 digest, hex:**  
```text  
{sha["digest_hex"]}
```
```

```
**h_sha256_mod_n:**  
{_code_block(sha["h_sha256_mod_n"])}  
`
```

```
**Подпись RSA/SHA-256:**  
{_code_block(sha["signature_integer"])}  
`
```

Подпись хранится в ASN.1 DER-контейнере `SignatureFile`.

```
**DER RSA/SHA-256, first 100 bytes, hex:**  
```text  
{_hex_block(sha["signature_der_first_100_hex"])}
```
```

Проверка исходного файла: **{sha["original_verification"]}**.
Проверка измененного файла: **{sha["modified_verification"]}**.

При проверке подписи хэш файла вычисляется заново, затем проверяется равенство $\text{signature}^e \bmod n == \text{SHA-256}(\text{file}) \bmod n$. Значение `hashValue` в контейнере используется как сохраненная диагностическая информация, но корректность подписи определяется пересчетом данных.

5. Дополнительное задание: RSA/CRC32

Для дополнительного задания вместо SHA-256 используется CRC32, после чего значение CRC32 подписывается RSA.

```
CRC32(D1):                **{crc["d1_crc32_hex"]}**          /
**{crc["d1_crc32_decimal"]}**.
CRC32(D2):                **{crc["d2_crc32_hex"]}**          /
**{crc["d2_crc32_decimal"]}**.
CRC32(D3):                **{crc["d3_crc32_hex"]}**          /
**{crc["d3_crc32_decimal"]}**.
```

```
**Подпись RSA/CRC32 для D1:**
{_code_block(crc["signature_integer"])}

```

Подпись RSA/CRC32 хранится в ASN.1 DER-контейнере `SignatureFile`.

```
**DER RSA/CRC32, first 100 bytes, hex:**
``text
{_hex_block(crc["signature_der_first_100_hex"])}
``

```

Проверка подписи D1: **{crc["d1_verification"]}**.

6. Демонстрация подбора D3

Файл D3 строится как `D3 = D2 || patch`, где `patch` - четыре байта, рассчитанные по линейности CRC32 над GF(2), без перебора 2^{32} значений.

```
Размер D2: **{crc["d2_size"]} байт**.
Размер D3: **{crc["d3_size"]} байт**.
Четырехбайтный patch, hex: **{crc["patch_hex"]}**.
```

```
D3 начинается с полного содержимого D2:  **{"OK"   if
crc["d3_starts_with_d2"] else "FAIL"}**.
Равенство CRC32(D3) и CRC32(D1):  **{"OK"   if
crc["d3_crc32_equals_d1"] else "FAIL"}**.
Проверка D3 старой подписью D1:
**{crc["d3_old_signature_verification"]}**.
```

7. Итоговые результаты проверки

- Генерация ключей RSA: **OK**
- Шифрование файла: **OK**
- Расшифрование файла: **OK**

```

-   Расшифрованный файл совпадает с исходным: **{"OK" if
files["decrypted_equals_original"] else "FAIL"}**
-   RSA/SHA-256 для исходного файла:
**{sha["original_verification"]}**
-   RSA/SHA-256 для измененного файла:
**{sha["modified_verification"]}**
-   RSA/CRC32 для D1: **{crc["d1_verification"]}**
-   CRC32 (D3) == CRC32 (D1): **{"OK" if crc["d3_crc32_equals_d1"] else
"FAIL"}**
-   D3 проходит проверку старой CRC32-подписью D1:
**{crc["d3_old_signature_verification"]}**
"""

```

```

def _build_verification_log(data: dict[str, Any]) -> str:
    # Краткий журнал нужен для приложения к отчету и быстрой
    проверки результатов.
    files = data["files"]
    sha = data["sha256_signature"]
    crc = data["crc32_signature"]
    lines = [
        "key generation OK",
        "encryption OK",
        "decryption OK",
        f"decrypted equals original {'OK' if
files['decrypted_equals_original'] else 'FAIL'}",
        f"SHA-256 signature original
{sha['original_verification']}",
        f"SHA-256 signature modified
{sha['modified_verification']}",
        f"CRC32 D1 signature {crc['d1_verification']}",
        f"CRC32 (D3) == CRC32 (D1) {'OK' if
crc['d3_crc32_equals_d1'] else 'FAIL'}",
        f"CRC32 D3 with old signature
{crc['d3_old_signature_verification']}",
    ]
    return "\n".join(lines) + "\n"

```

```

def _build_hex_dump(data: dict[str, Any]) -> str:
    aes = data["aes"]
    files = data["files"]
    return f"""# Hex dump для Lab1

```

```

## Первые 100 байт DER-контейнера

```

```

```text
{_hex_block(files["container_first_100_hex"])}
```

## DER RSA/SHA-256 signature container, first 100 bytes

```text
{_hex_block(data["sha256_signature"]["signature_der_first_100_hex"
])}]
```

## DER RSA/CRC32 signature container, first 100 bytes

```text
{_hex_block(data["crc32_signature"]["signature_der_first_100_hex"
])}]
```

## Первые 100 байт AES ciphertext

```text
{_hex_block(files["ciphertext_first_100_hex"])}
```

## AES-256 ключ

```text
{_hex_block(aes["aes_key_hex"])}
```

## IV

```text
{aes["iv_hex"]}
```

## Зашифрованный AES-ключ

```text
{_hex_block(aes["encrypted_key_hex"])}
```

"""

def _build_crc32_demo(data: dict[str, Any]) -> str:
    crc = data["crc32_signature"]

```

```

return f"""# Демонстрация дополнительного задания RSA/CRC32

CRC32(D1):                **{crc["d1_crc32_hex"]}**                /
**{crc["d1_crc32_decimal"]}**.

CRC32(D2):                **{crc["d2_crc32_hex"]}**                /
**{crc["d2_crc32_decimal"]}**.

Четырехбайтный patch, hex: **{crc["patch_hex"]}**.

CRC32(D3):                **{crc["d3_crc32_hex"]}**                /
**{crc["d3_crc32_decimal"]}**.

D3 построен как `D3 = D2 || patch`.

D3 сохраняет все содержимое D2: **{"OK" if crc["d3_starts_with_d2"]
else "FAIL"}**.

D3 проходит проверку со старой CRC32-подписью D1:
**{crc["d3_old_signature_verification"]}**.
"""

def _build_theory() -> str:
    return """# Теоретические сведения

## 1. RSA

В RSA выбираются два простых числа `p` и `q`, после чего вычисляется
модуль `n = p * q`. Функция Эйлера для такого модуля равна `phi(n) =
(p-1)(q-1)`. Открытый показатель `e` выбирается взаимно простым с
`phi(n)`, а закрытый показатель `d` удовлетворяет сравнению `e * d ≡
1 mod phi(n)`.

Шифрование целого представителя сообщения выполняется как `c = m^e
mod n`, расшифрование как `m = c^d mod n`. В лабораторной
используется учебная реализация RSA для демонстрации принципов. В
реальных системах textbook RSA без padding не применяют: для
шифрования нужен OAEP, для подписи - PSS или другой
стандартизованный padding.

## 2. Гибридное шифрование RSA + AES

RSA не используют для шифрования больших файлов напрямую: операция
медленная, размер блока ограничен модулем, а без padding схема
детерминирована. Поэтому применяется гибридная схема: данные файла

```

шифруются симметричным алгоритмом AES, а RSA шифрует только случайный AES-ключ.

В данной работе файл шифруется AES-256-CBC. Случайный 32-байтный ключ AES помещается в контейнер в RSA-зашифрованном виде. IV нужен для режима CBC, чтобы одинаковые первые блоки открытого текста не давали одинаковые первые блоки шифртекста при одном и том же ключе.

3. AES-256-CBC

AES-256 использует ключ длиной 256 бит и размер блока 128 бит. Режим CBC связывает каждый блок открытого текста с предыдущим блоком шифртекста через XOR перед шифрованием. Для первого блока используется IV длиной 16 байт.

Так как AES работает с блоками фиксированного размера, перед шифрованием применяется PKCS#7 padding. При расшифровании padding проверяется и удаляется, после чего восстанавливается исходный файл.

4. ASN.1 DER-контейнер

Структурированный бинарный контейнер нужен, чтобы хранить вместе версию формата, RSA-зашифрованный ключ, IV, шифртекст и параметры подписи без неоднозначного разбора. DER является однозначным бинарным кодированием ASN.1.

Зашифрованный файл хранится как:

```
```text
EncryptedFile ::= SEQUENCE {
 version INTEGER,
 encryptedKey OCTET STRING,
 iv OCTET STRING,
 ciphertext OCTET STRING
}
```
```

Файл подписи хранится как:

```
```text
SignatureFile ::= SEQUENCE {
 version INTEGER,
 algorithm UTF8String,
 hashValue OCTET STRING,
 hashModN INTEGER,

```

```

signature INTEGER
}
...

5. Электронная подпись RSA/SHA-256

Для подписи вычисляется $h = \text{SHA-256}(\text{file}) \bmod n$, затем $s = h^d \bmod n$. Проверка выполняет обратную операцию с открытым ключом: $s^e \bmod n == h$. SHA-256 подходит для лабораторной как криптографическая хэш-функция с большим выходом и практической устойчивостью к подбору коллизий.

6. CRC32 и почему он не подходит для электронной подписи

CRC32 является линейной контрольной суммой для обнаружения случайных ошибок, а не криптографическим хэшем. Выход имеет всего 32 бита, коллизии легко находятся, а линейность позволяет конструктивно изменять данные при сохранении заданного CRC32.

В дополнительном задании строится файл $D3 = D2 \parallel \text{patch}$, где четыре байта patch подбираются так, чтобы $\text{CRC32}(D3) == \text{CRC32}(D1)$. Если подпись строится по CRC32, то D3 проходит проверку старой подписью D1, хотя содержимое файла другое.
"""

def _build_control_questions() -> str:
 return """# Контрольные вопросы

1. Почему RSA не шифрует весь файл напрямую?

RSA медленнее симметричных алгоритмов и может обрабатывать только данные меньше модуля n . Кроме того, textbook RSA без padding небезопасен. Поэтому RSA шифрует только ключ, а файл шифруется AES.

2. Зачем используется AES-256-CBC в гибридной схеме?

AES быстро шифрует данные произвольного размера, а ключ длиной 256 бит дает достаточный запас стойкости для учебной реализации. RSA защищает только AES-ключ.

3. Зачем нужен IV в CBC?

IV задает начальное значение цепочки CBC. Он делает шифрование первого блока зависимым от случайного значения, поэтому одинаковые сообщения при одном ключе не начинаются одинаковым шифртекстом.

```

## 4. Что хранится в ASN.1 DER-контейнере зашифрованного файла?

Версия формата, RSA-зашифрованный AES-ключ, IV и AES-шифртекст файла.

## 5. Что хранится в ASN.1 DER-контейнере подписи?

Версия, имя алгоритма, сохраненное значение хэша, хэш как число по модулю  $n$  и целое значение RSA-подписи.

## 6. Как формируется RSA/SHA-256 подпись?

Сначала вычисляется SHA-256 от файла, затем хэш переводится в целое число и приводится по модулю  $n$ . Подпись равна  $s = h^d \bmod n$ .

## 7. Как проверяется RSA/SHA-256 подпись?

Проверяющая сторона заново вычисляет SHA-256 файла, получает  $h$ , затем проверяет, что  $s^e \bmod n == h$ .

## 8. Почему при проверке нельзя доверять hashValue из контейнера без пересчёта?

Контейнер может быть изменен вместе с файлом. Если не пересчитывать хэш по реальным данным, проверка подтвердит не содержимое файла, а только сохраненное в контейнере значение.

## 9. Чем CRC32 отличается от SHA-256?

CRC32 – короткая линейная контрольная сумма для ошибок передачи. SHA-256 – криптографическая хэш-функция с 256-битным выходом, рассчитанная на сопротивление коллизиям и подделке.

## 10. Почему CRC32 нельзя использовать как криптографический хэш для ЭП?

Из-за 32-битного размера и линейности CRC32 легко построить другое сообщение с тем же значением. Такая подпись не защищает целостность от намеренной подделки.

## 11. Как работает подбор  $D3 = D2 \parallel \text{patch}$ ?

К выбранному содержимому  $D2$  добавляются четыре специально рассчитанных байта. Из-за линейных свойств CRC32 эти байты переводят итоговый CRC32 в заранее заданное значение CRC32( $D1$ ).

## 12. Почему D3 проходит проверку старой подписью D1?

Потому что подпись RSA/CRC32 проверяет только значение CRC32. Если ``CRC32(D3) == CRC32(D1)``, то старая подпись от D1 подтверждает и D3.

## 13. Почему в реальных системах RSA-подпись должна использовать padding, например PSS?

Padding устраняет опасные алгебраические свойства textbook RSA, добавляет случайность и структуру проверки. PSS является стандартизированной схемой RSA-подписи с доказуемыми свойствами безопасности.

"""

```
def _build_stand_commands() -> str:
 return r"""# Команды для защиты
```

## Проверка лабораторной

```
```powershell
cd C:\Users\Georgul\Documents\8_sem\KMZI\Lab1
.\.venv\Scripts\Activate.ps1
pytest -q
python scripts\generate_report_materials.py
Get-Content .\report\generated\verification_log.txt -Encoding UTF8
```
```

## Основная CLI-демонстрация

```
```powershell
python -m src.main keygen --bits 1024 --out artifacts
python -m src.main encrypt --file data\message.txt --public-key
artifacts\public.json --out artifacts\message.enc --debug-json
artifacts\encrypt_debug.json
python -m src.main decrypt --file artifacts\message.enc --private-
key artifacts\private.json --out artifacts\message.dec
python -m src.main sign-sha256 --file data\message.txt --private-
key artifacts\private.json --signature
artifacts\message.sha256.sig
python -m src.main verify-sha256 --file data\message.txt --public-
key artifacts\public.json --signature artifacts\message.sha256.sig
python -m src.main sign-crc32 --file data\d1.txt --private-key
artifacts\private.json --signature artifacts\d1.crc32.sig
```

```
python -m src.main verify-crc32 --file data\d1.txt --public-key
artifacts\public.json --signature artifacts\d1.crc32.sig
python -m src.main forge-crc32 --original data\d1.txt --modified
data\d2.txt --out artifacts\d3.txt
python -m src.main verify-crc32 --file artifacts\d3.txt --public-
key artifacts\public.json --signature artifacts\d1.crc32.sig
```
```

## Проверка первого байта DER

```
```powershell
Format-Hex .\artifacts\message.enc | Select-Object -First 1
Format-Hex .\artifacts\message.sha256.sig | Select-Object -First 1
Format-Hex .\artifacts\d1.crc32.sig | Select-Object -First 1
```
```

Ожидаемый первый байт: `30`, что соответствует ASN.1 DER  
`SEQUENCE`.

"""

```
def _build_full_report_draft(report_data: str, theory: str,
control_questions: str) -> str:
 theory_body = theory.removeprefix("# Теоретические
сведения\n\n")
 practical_body = report_data.removeprefix("# Данные для отчета
по Lab1\n\n")
 control_body = control_questions.removeprefix("# Контрольные
вопросы\n\n")
 return f"""# Лабораторная работа №1. Основы RSA
```

## Цель работы

Изучить принципы RSA, гибридного шифрования RSA + AES-256-CBC, ASN.1 DER-контейнеров и электронной подписи RSA/SHA-256, а также показать непригодность CRC32 для криптографической подписи.

## Задание

Реализованы генерация ключей RSA, шифрование и расшифрование файла по гибридной схеме, сохранение данных в DER-контейнере, создание и проверка подписи RSA/SHA-256, подпись RSA/CRC32 и демонстрация подбора файла D3 с тем же CRC32, что и у D1.

## Теоретические сведения

```

{theory_body}

Описание реализации

Код разделен на модули: `rsa.py` реализует RSA, `aes.py` - AES-256-CBC и padding, `der.py` и `sigder.py` - DER-контейнеры, `hybrid.py` - файловое гибридное шифрование, `signatures.py` - подписи, `crcforge.py` - подбор четырехбайтного CRC32-патча. CLI в `src/main.py` запускает основные операции лабораторной.

Практические результаты

{practical_body}

Дополнительное задание

Дополнительное задание демонстрирует слабость подписи по CRC32. Файл D3 строится как `D3 = D2 || patch`, где `patch` рассчитан так, чтобы `CRC32(D3) == CRC32(D1)`. Поэтому подпись, созданная для D1, проходит проверку для D3, хотя содержимое D3 начинается с выбранного D2.

Контрольные вопросы

{control_body}

Вывод

В работе реализована полная учебная схема защиты файла: RSA используется для ключей и подписи, AES-256-CBC - для данных, DER - для структурированного хранения. Проверка RSA/SHA-256 корректно отвергает измененный файл. Демонстрация CRC32 показывает, что контрольные суммы нельзя использовать как криптографические хэши для электронной подписи.
"""

def generate_report_materials(lab_root: Path | None = None, bits: int = 1024) -> dict[str, Any]:
 """Сформировать материалы отчета и вернуть собранные данные."""
 root = (lab_root or DEFAULT_LAB_ROOT).resolve()
 if str(root) not in sys.path:
 sys.path.insert(0, str(root))

 data_dir = root / "data"

```

```

artifacts_dir = root / "artifacts" / "report_run"
generated_dir = root / "report" / "generated"
Временные артефакты очищаются, а report/generated остается
папкой
с готовыми для вставки в отчет материалами.
_clean_dir(artifacts_dir)
generated_dir.mkdir(parents=True, exist_ok=True)

message_path = data_dir / "message.txt"
d1_path = data_dir / "d1.txt"
d2_path = data_dir / "d2.txt"

1. Генерация RSA-ключей и сохранение параметров p, q, n, e,
d.
private_key, public_key = generate_keypair(bits)
private_key_path = artifacts_dir / "private.json"
public_key_path = artifacts_dir / "public.json"
save_private_key(private_key_path, private_key)
save_public_key(public_key_path, public_key)

encrypted_path = artifacts_dir / "message.enc"
decrypted_path = artifacts_dir / "message.dec"
debug_json_path = artifacts_dir / "encrypt_debug.json"
2. Гибридное шифрование файла и debug JSON с AES/DER
значениями.
encrypt_file(message_path, public_key, encrypted_path,
debug_json_path)
3. Расшифрование и байтовое сравнение с исходным файлом.
decrypt_file(encrypted_path, private_key, decrypted_path)

message = message_path.read_bytes()
decrypted_equals_original = decrypted_path.read_bytes() ==
message
debug = json.loads(debug_json_path.read_text(encoding="utf-
8"))
container_bytes = encrypted_path.read_bytes()

sha_signature_path = artifacts_dir / "message.sha256.sig"
4. RSA/SHA-256 подпись: проверяем исходный файл и
отрицательный пример.
sha_container = create_sha256_container(message, private_key)
save_signature(sha_signature_path, sha_container)
sha_loaded = load_signature_container(sha_signature_path)
sha_original_valid = verify_sha256_container(message,
sha_loaded, public_key)
modified_path = artifacts_dir / "message.modified.txt"

```

```

 modified_message = message + b"\nmodified for negative
signature check\n"
 modified_path.write_bytes(modified_message)
 sha_modified_valid =
verify_sha256_container(modified_message, sha_loaded, public_key)
 sha_digest = hashlib.sha256(message).digest()
 sha_signature_der = sha_signature_path.read_bytes()

 crc_signature_path = artifacts_dir / "d1.crc32.sig"
 d1_bytes = d1_path.read_bytes()
 d2_bytes = d2_path.read_bytes()
 # 5. RSA/CRC32 подпись для D1 как часть дополнительного
задания.
 crc_container = create_crc32_container(d1_bytes, private_key)
 save_signature(crc_signature_path, crc_container)
 crc_loaded = load_signature_container(crc_signature_path)
 crc_d1_valid = verify_crc32_container(d1_bytes, crc_loaded,
public_key)
 crc_signature_der = crc_signature_path.read_bytes()

 d3_path = artifacts_dir / "d3.txt"
 # 6. Построение D3 = D2 || patch без перебора 2^32 вариантов.
 forge_crc32_file(d1_path, d2_path, d3_path)
 d3_bytes = d3_path.read_bytes()
 patch = d3_bytes[len(d2_bytes) :]
 crc_d3_old_signature_valid = verify_crc32_container(d3_bytes,
crc_loaded, public_key)

 d1_crc = crc32_bytes(d1_bytes)
 d2_crc = crc32_bytes(d2_bytes)
 d3_crc = crc32_bytes(d3_bytes)

 data: dict[str, Any] = {
 "rsa": {
 "p": str(private_key.p),
 "q": str(private_key.q),
 "n": str(private_key.n),
 "e": str(private_key.e),
 "d": str(private_key.d),
 "modulus_bit_length": private_key.n.bit_length(),
 },
 "aes": {
 "aes_key_hex": debug["aes_key_hex"],
 "iv_hex": debug["iv_hex"],
 "encrypted_key_hex": debug["encrypted_key_hex"],
 },
 },

```

```

 "files": {
 "original_message_size": len(message),
 "encrypted_container_size": len(container_bytes),
 "decrypted_equals_original":
decrypted_equals_original,
 "container_first_100_hex":
debug["container_first_100_hex"],
 "ciphertext_first_100_hex":
debug["ciphertext_first_100_hex"],
 },
 "sha256_signature": {
 "digest_hex": sha_digest.hex(),
 "h_sha256_mod_n": str(sha_loaded.hash_mod_n),
 "signature_integer": str(sha_loaded.signature),
 "signature_der_first_100_hex":
sha_signature_der[:100].hex(),
 "original_verification": _status(sha_original_valid),
 "modified_verification": _status(sha_modified_valid),
 },
 "crc32_signature": {
 "d1_crc32_hex": f"{d1_crc:08x}",
 "d1_crc32_decimal": d1_crc,
 "d2_crc32_hex": f"{d2_crc:08x}",
 "d2_crc32_decimal": d2_crc,
 "d3_crc32_hex": f"{d3_crc:08x}",
 "d3_crc32_decimal": d3_crc,
 "patch_hex": patch.hex(),
 "d2_size": len(d2_bytes),
 "d3_size": len(d3_bytes),
 "d3_starts_with_d2": d3_bytes.startswith(d2_bytes),
 "d3_crc32_equals_d1": d3_crc == d1_crc,
 "signature_integer": str(crc_loaded.signature),
 "signature_der_first_100_hex":
crc_signature_der[:100].hex(),
 "d1_verification": _status(crc_d1_valid),
 "d3_old_signature_verification":
_status(crc_d3_old_signature_valid),
 },
 }

 report_data = _build_report_markdown(data)
 theory = _build_theory()
 control_questions = _build_control_questions()
 stand_commands = _build_stand_commands()
 full_report = _build_full_report_draft(report_data, theory,
control_questions)

```

```

 # 7. Запись машинно-читаемых и человеко-читаемых файлов отчета.
 _write_json(generated_dir / "report_data.json", data)
 (generated_dir / "report_data.md").write_text(report_data,
encoding="utf-8")
 (generated_dir /
"verification_log.txt").write_text(_build_verification_log(data),
encoding="utf-8")
 (generated_dir /
"hex_dump.md").write_text(_build_hex_dump(data), encoding="utf-
8")
 (generated_dir /
"crc32_demo.md").write_text(_build_crc32_demo(data),
encoding="utf-8")
 (generated_dir / "theory.md").write_text(theory,
encoding="utf-8")
 (generated_dir /
"control_questions.md").write_text(control_questions,
encoding="utf-8")
 (generated_dir /
"stand_commands.md").write_text(stand_commands, encoding="utf-8")
 (generated_dir /
"full_report_draft.md").write_text(full_report, encoding="utf-8")

 return data

def main() -> int:
 generate_report_materials()
 print("Generated report materials:")
 for filename in REPORT_FILES:
 print(f"- report/generated/{filename}")
 return 0

if __name__ == "__main__":
 raise SystemExit(main())

```